AI-powered Virtual Humans

Part 1: Virtual Humans Foundations



Day 1

Stefano Calzolari · Wed 21 – Fri 23 October 2026

01

Virtual Humans and their applications

What are virtual humans, and what are not?

Where are they used for, and why they matter?







Images: Machidon et al., 2018 (Pompeii) · DeVault et al., SimSensei Kiosk (Ellie) · Tepe et al., 2012, in Eiris & Gheisari, 2017 (construction)

What are Virtual Humans?

“Artificial characters who look and act like humans but inhabit a simulated environment.”

What should they be able to do?

- **Communicate with the user, verbally and not**
- **Understand the environment and the user's intentions**
- **Interact with the environment itself**



Max, a virtual human, and a user in a VR setup. Leßmann, Kopp & Wachsmuth, 2006

Traum (2008), in Lugrin, The Handbook on Socially Interactive Agents, ACM Books, 2021

Situated interaction: sharing the user's world

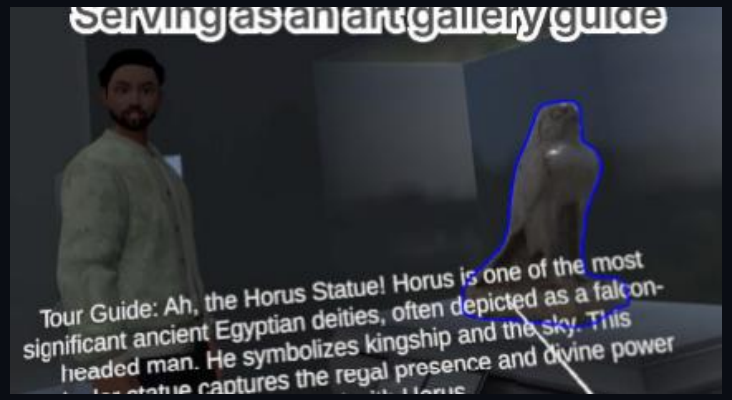
"The conversation is an embodied interaction between agents, where language, gesture, gaze, and actions are situated within a common ground shared by all agents." Pustejovsky & Krishnaswamy, 2020

"Cohabiting a virtual environment with the human users, virtual agents appear as equal partners that share the very situation with the user." Leßmann, Kopp & Wachsmuth, 2006



1 Communicate

Speech, gaze, gesture and face, in sync



2 Understand

The scene, and what the user means: words, pointing, gaze



3 Act

Move, pick up, hand over: change the shared world

Pustejovsky & Krishnaswamy, Embodied human-computer interactions through situated grounding, IVA 2020 · Leßmann, Kopp & Wachsmuth, Situated interaction with a virtual human, in Situated Communication, 2006 · Images: Shoa et al., IVA 2023 · Li et al., IEEE VR 2025 · Pecora, Calzolari et al., A.D.A.M.O., 2026

Glossary: terms of the field

Term	Meaning
Agent	Perceives through sensors, acts through actuators (Russell & Norvig)
Avatar	Representation of a user in a virtual world; not autonomous
Virtual character	A figure plus its animations; not necessarily intelligent or autonomous
Virtual Human (VH)	Looks and acts like a human in a simulated environment (Traum)
Intelligent Virtual Agent (IVA)	Interactive agent with perception, cognition, emotion and action
Embodied Conversational Agent (ECA)	Agent with a body that converses with speech, gaze and gestures

- **Virtual Human** is our general term, especially for XR
- **IVA** and **ECA** are easy to find in literature
- In this course we will build an **Embodied Conversational Agent (ECA)**

Lugrin, Introduction to Socially Interactive Agents, in The Handbook on Socially Interactive Agents, ACM Books, 2021

Cultural Heritage



Crowd

Give life to a virtual space



Guide

Senses visitors and interacts



Protagonist

The character enacts the story

Machidon, Duguleana & Carrozzino, Virtual humans in cultural heritage ICT applications: a review, Journal of Cultural Heritage, 2018 (Fig. 1d) · Sylaiou & Fidas, Virtual Humans in Museums and Cultural Heritage Sites, Applied Sciences, 2022 (Fig. 11, © Uni. Geneva/MIRALab) · Museo Egizio Interattivo, Robin, RIBES DigiLab, Politecnico di Torino, Museo Egizio, CHANGES

Architecture, Engineering and Construction



Safety and education

Spot dangers on a site that cannot hurt you



System evaluation and analysis

Test reach, visibility and evacuation with people inside



Scheduling

Virtual workers play out the schedule step by step



Collaboration and communication

Avatars meet around the same building model

Eiris & Gheisari, Research trends of virtual human applications in architecture, engineering and construction, Journal of Information Technology in Construction (ITcon), 2017 (Fig. 8, 9, after Tepe 2012, Woksepp & Olofsson 2006, Sacks 2014, Anderson & Dossick 2014)

Healthcare



Screening

- Listens with nods and open questions
- Reads face, gaze and voice for the clinician
- People disclose more when no human judges





Training with virtual patients

- Stand in for patient actors: always available, always the same
- Any patient: a child, an elder, a difficult conversation
- History taking and bad news, now driven by LLMs in VR

DeVault et al., SimSensei Kiosk · Talbot & Rizzo, Virtual human standardized patients for clinical training, 2019 (Fig. 17.8) · Zhu et al., VR clinical communication training with LLM-based ECAs, CHI EA 2025 (Fig. 1) · Lee et al., Medical Education, 2020

InCare: communication training in VR



Military



Mission rehearsal

Lead under stress when civilians are hurt



Negotiation

Persuade a doctor who does not trust you



Leadership

New officers counsel a virtual sailor



Care for veterans

Clinicians practise with a virtual soldier

Swartout et al., Toward virtual humans, AI Magazine, 2006 (Fig. 1, 4) · Campbell et al., Developing INOTS, 2011 (Fig. 3) · Talbot & Rizzo, 2019 (Fig. 17.3)

Media, Commerce and Entertainment



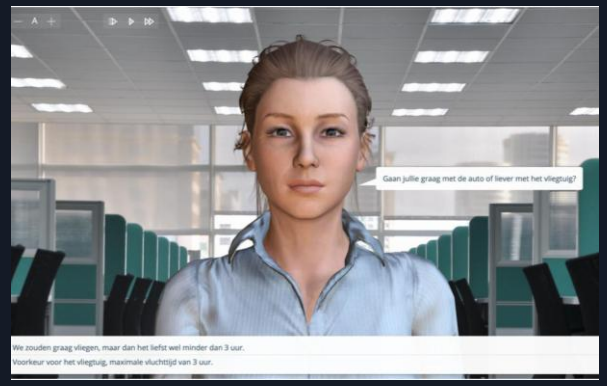
Virtual hosts

A digital presenter runs the show: news, interviews, ads



Influencers and idols

A brand face that never ages or goes off script



Customer service

Service agents that answer, advise and build rapport



Games

Characters that talk back and give quests

Seymour et al., JAIS 2021 (MEETMIKE) · Mirowska & Arsenyan 2023 · van Pinxteren et al., Psychology & Marketing 2023 · Rao et al. 2024 · Cui & Liu, IEEE Access 2023

LLM-based Skyrim Merchant



02

Believability: requirements for virtual humans



What makes a virtual human believable?

Where does the illusion break?

Images: Seymour, Yuan, Dennis & Riemer, JAIS, 2021 · AlJammaz, Wardrip-Fruin & Mateas, FDG, 2023

Believability: the definition we use

> *"Believability refers to the extent to which people believe that the agent they observe and interact with is an autonomous, sentient being."*
>
> Guo, Adamo & Mousas, 2023, after Lester & Stone, 1997

Illusion of life

From Disney to Bates (1994): the audience willingly suspends disbelief.

Not realism

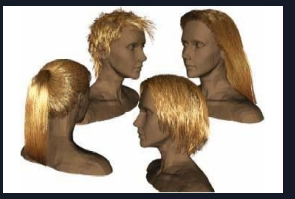
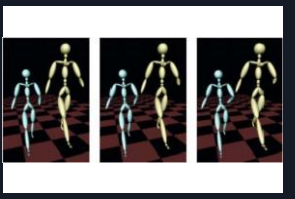
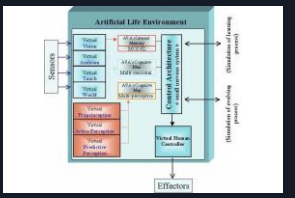
Realism helps, but a believable character need not be realistic.

Multidimensional

It emerges from a system that integrates many aspects at once.

Guo, Adamo & Mousas, Developing a scale for measuring the believability of virtual agents, ICAT-EGVE 2023 · Lester & Stone, 1997 · Bates, The role of emotion in believable agents, CACM, 1994 · Mateas, 2001

Three levels of modeling

Appearance	Body shape, skin, hair, clothes How the virtual human looks	
Motion	Kinematics, mocap, idle motion *“realistic, smooth and flexible motion in any situation”* How it moves	
Cognition	Perception, autonomy, emotion, personality How it perceives, decides and feels	

Magnenat-Thalmann & Thalmann, Virtual humans: thirty years of research, what next?, The Visual Computer, 2005

Believability dimensions

Guo et al. order them from static, low level to dynamic, high level: they fall onto Thalmann's three levels

APPEARANCE

Visual properties

Face and body, realism, visual impact; sets what users expect

MOTION

Behavior

Coherent, natural, fit to the context; predictable, yet not robotic

COGNITION

Awareness

Of the environment, of itself, of the interaction

Social relationships

Aware of others, fits the social context

Intelligence

Plans, learns, adapts, remembers

Emotion

Expresses fitting, readable emotions

Personality

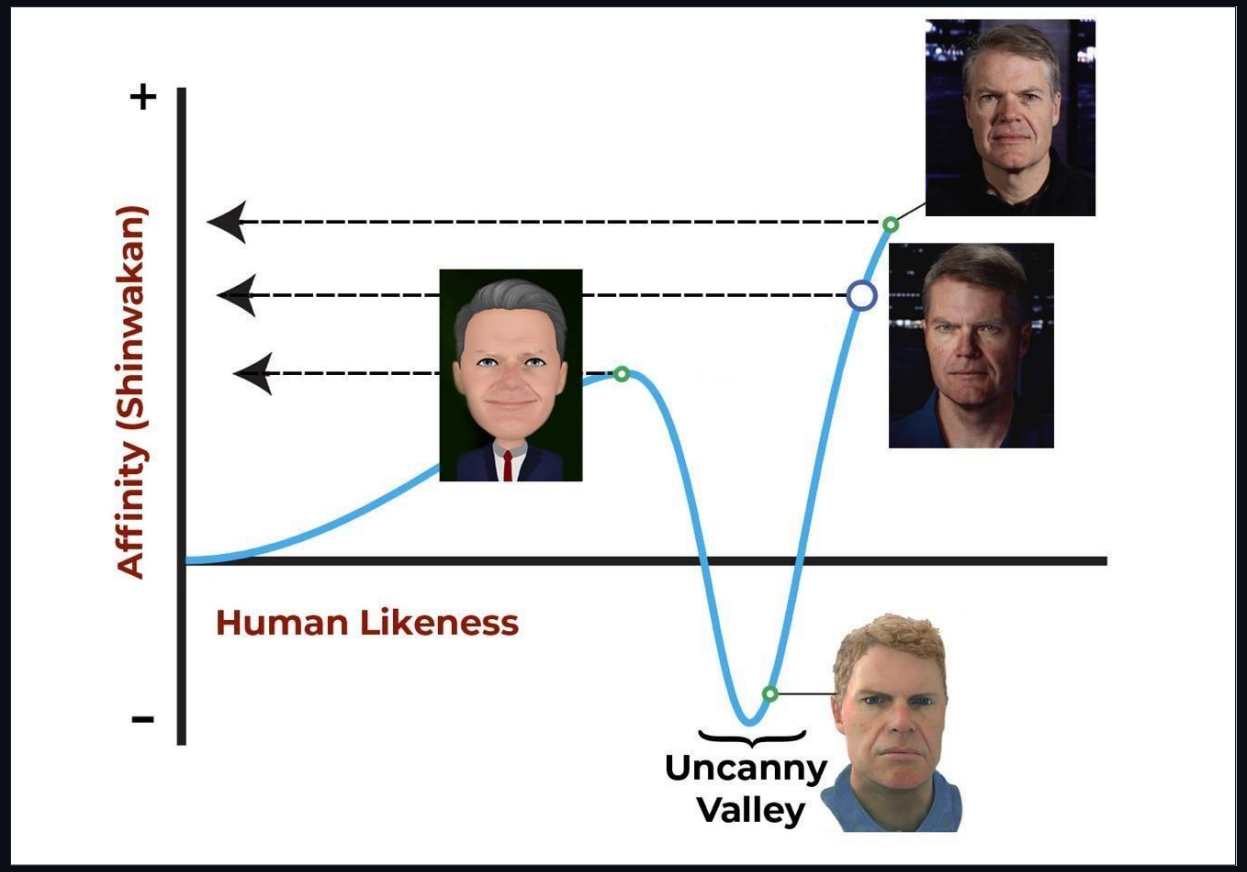
A pattern of behavior that stays coherent

Agency

Has goals, acts on its own

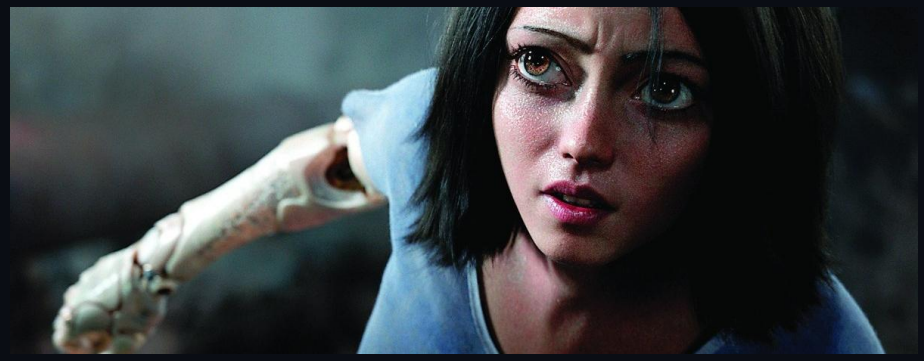
Guo, Adamo & Mousas, Developing a scale for measuring the believability of virtual agents, ICAT-EGVE 2023 (Sec. 2) · Magnenat-Thalmann & Thalmann, 2005

Visual properties: the Uncanny Valley



Where it breaks

- No idle life
- Mouth and voice out of sync
- Eyes that do not move
- An answer two seconds late



Seymour, Yuan, Dennis & Riemer, Have we crossed the uncanny valley?, JAIS, 2021 (Fig. 9) · Mori, The uncanny valley, 1970 · Schwind, Wolf & Henze, Avoiding the uncanny valley in virtual character design, Interactions, 2018

Believability in practice: what studies show

Dimension	What the studies found	Studies
Appearance	Realistic faces rarely seem uncanny; natural voices help.	Seymour 2021 · Zibrek 2019 · Higgins 2021, 2023 · Volonte 2016 · Jun 2020 · Parmar 2022
Behavior	Behavior weighs most: gaze, distance and quick replies.	Groom 2009 · Kolkmeier 2016 · Christiansen 2024
Awareness	An agent that shows it knows its surroundings raises presence and trust.	Ijaz 2011 · Kim 2018 · Kontogiorgos 2019
Intelligence	Agents that remember, plan and adapt to the user seem more natural.	Park 2023 · Woo 2024
Social relationships	Timely nods and social cues build rapport.	Gratch 2007 · Pejsa 2017 · Van Pinxteren 2023 · Li 2023 · Jeong 2025
Emotion	Emotion adds little on its own, but it drives empathy and rapport.	Higgins 2023 · Hoegen 2018 · Kroes 2022
Personality	Personality is read from motion and voice.	Thomas 2022 · Sajjadi 2019 · Zhou 2019 · Sonlu 2025 · Kroczek 2025
Agency	People treat an agent that acts on purpose more like a person.	de Melo & Gratch 2015 · Gilbert & Forney 2015

Guo, Adamo & Mousas, ICAT-EGVE 2023 (dimensions) · Other studies by first author and year in the table

03

Classical Architectures of Virtual Humans

How are virtual humans built?

What is the logic driving them?







Images: Swartout et al., AI Magazine, 2006 · Kopp, Jung, Leßmann & Wachsmuth, KI, 2003 · Hartholt et al., HAI, 2019

SAIBA: from intent to motion

Intent planning

What to communicate, and when to take the turn

↓ **FML** the meaning: intent, emotion, turn, grounding

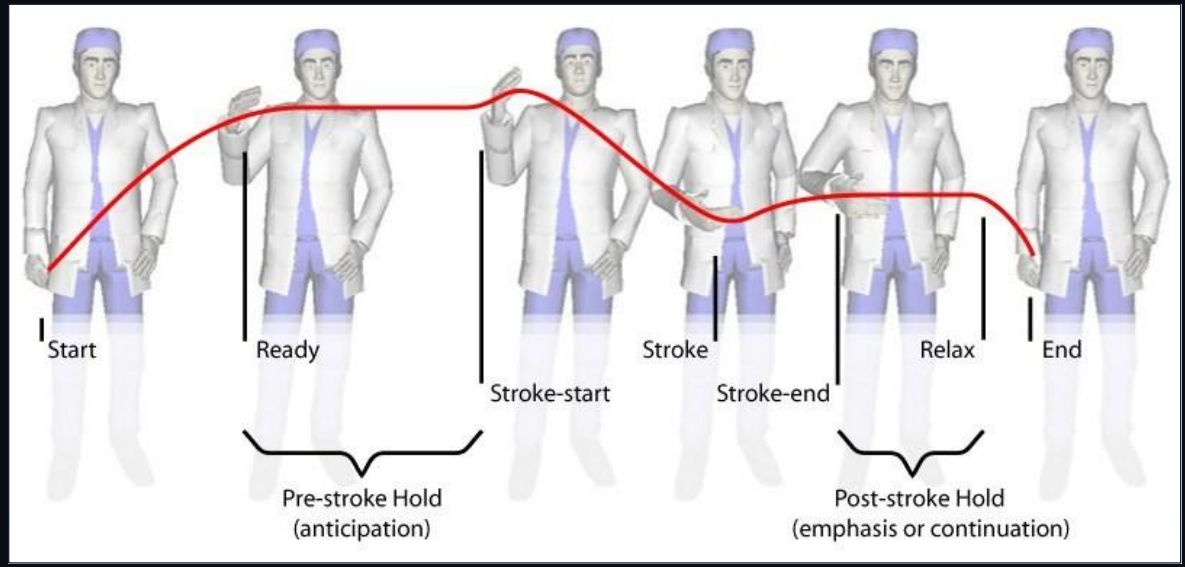
Behavior planning

Which words, gaze, gestures, face

↓ **BML** the signals and their sync points

Behavior realization

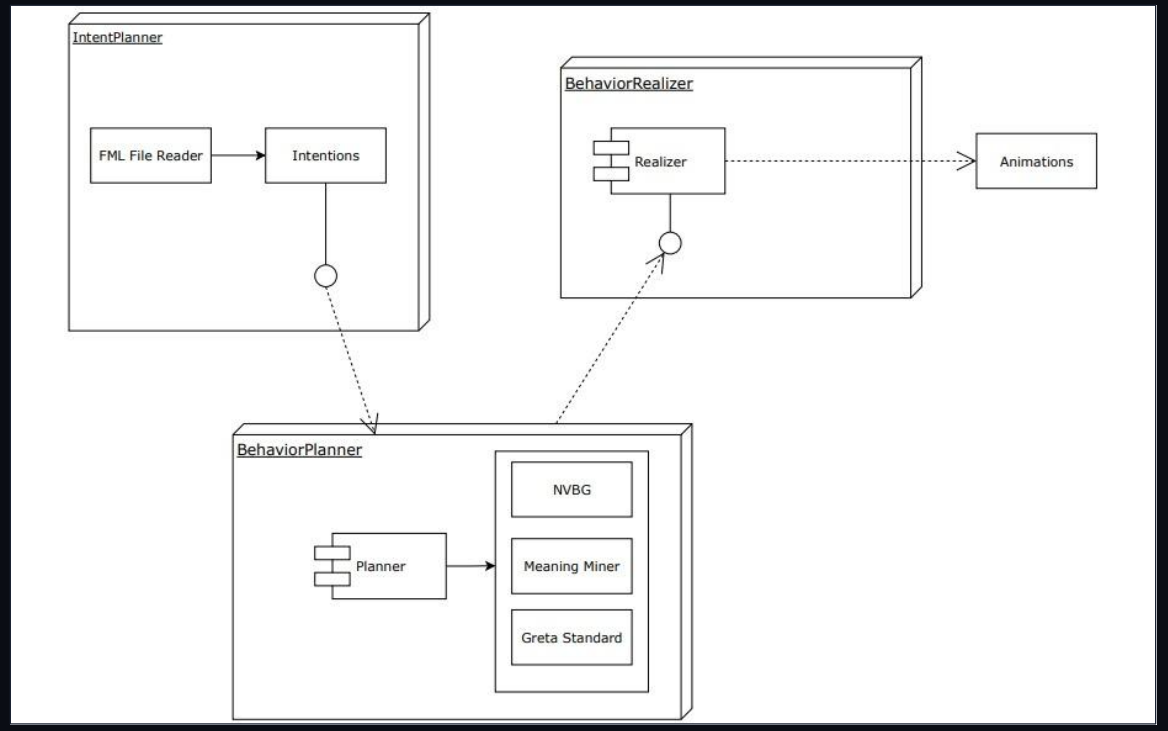
The engine renders it, in sync



BML sync points of a gesture: start, ready, stroke-start, **stroke**, stroke-end, relax, end. Speech, gaze and face align on the same points.

Kopp et al., IVA 2006 (Fig. 3) · Vilhjálmsson et al., IVA 2007 · Heylen et al., IVA 2008 · Bevacqua et al., AAMAS 2009

Greta: the reference SAIBA agent

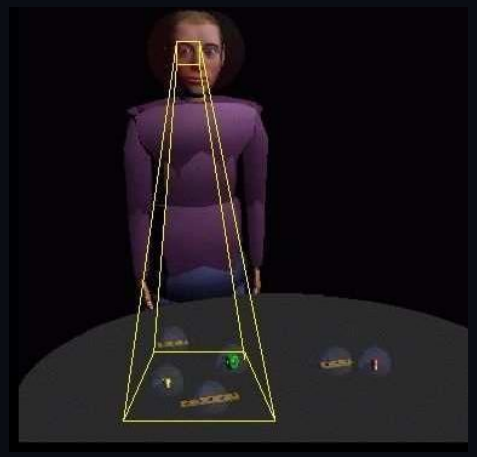




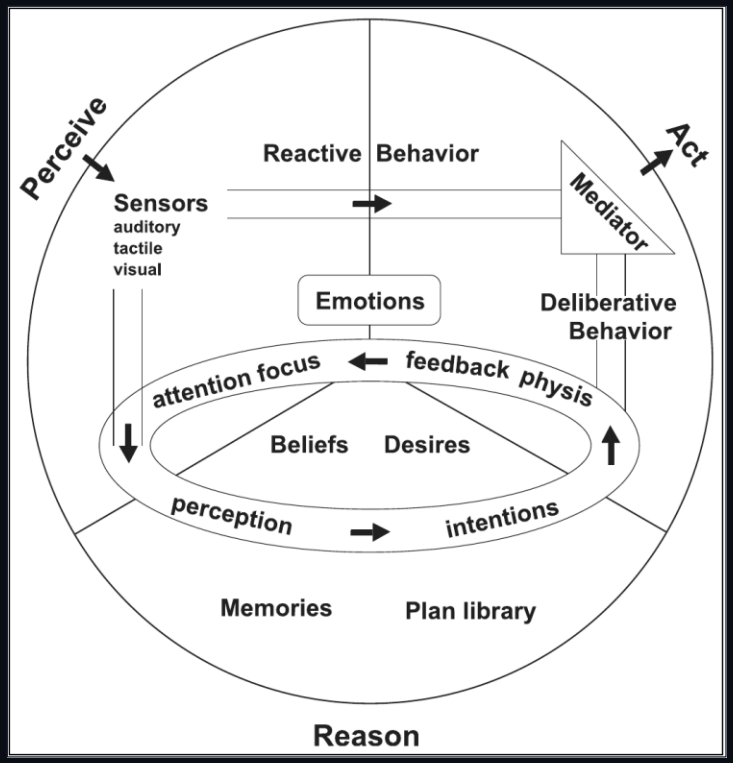
Grimaldi et al., GRETA: modular platform · Bevacqua et al., AAMAS 2009

Max: an agent in a shared world





Left: the user points while Max builds with him. Right: what Max sees, his field of view

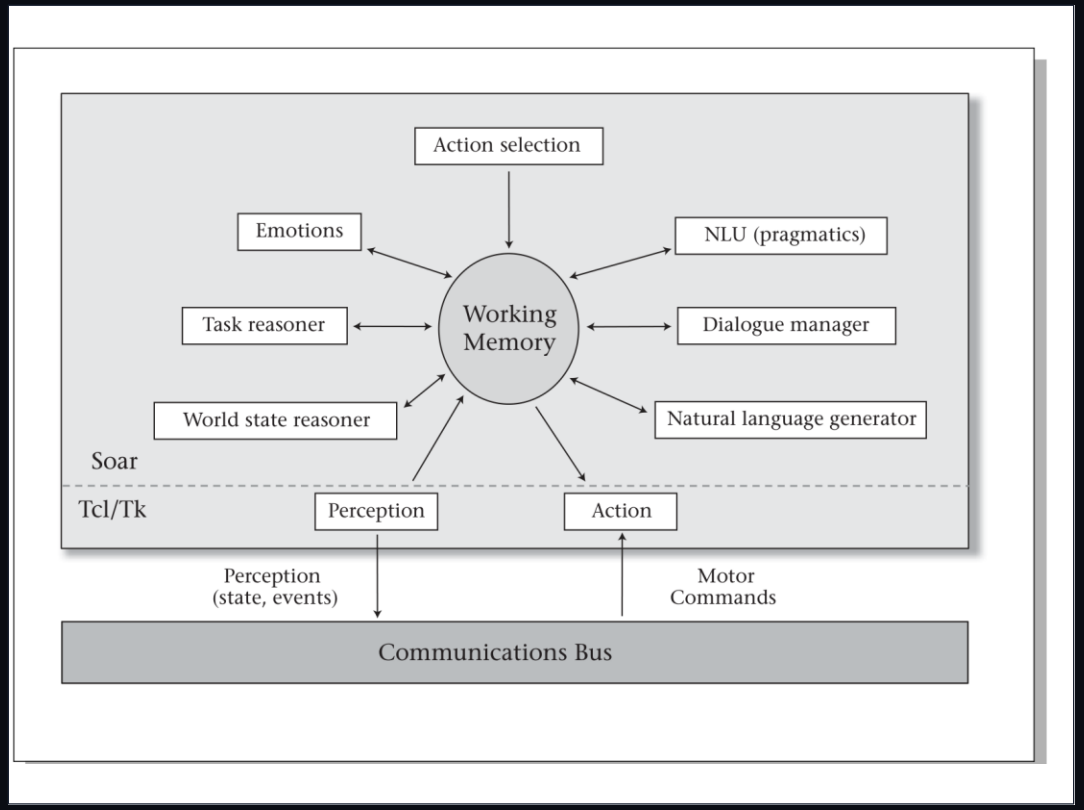


Perceive, reason, act: a fast reactive path and a slower deliberative one meet in a mediator; emotions influence both

Leßmann, Kopp & Wachsmuth, Situated Communication, 2006 · Kopp et al., KI, 2003 · Becker-Asano, WASABI, PhD thesis, 2008 (Fig. 1.3)

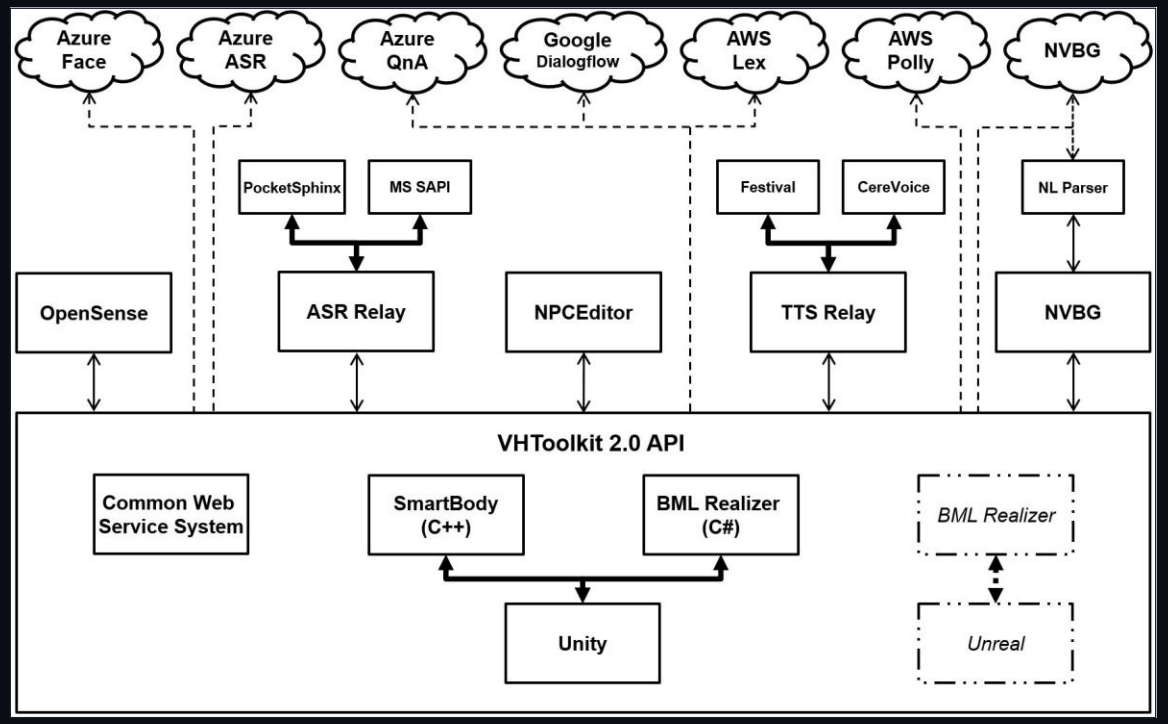
MRE: emotion, task and dialogue together





Swartout, Gratch, Marsella et al., Toward virtual humans, AI Magazine, 2006 (Fig. 2) · Marsella & Gratch, EMA: a process model of appraisal dynamics

VHToolkit 2.0: from monolith to services



What made it unique

Services	Sensing, cognition, voice and engine talk through narrow messages
Still SAIBA	FML, BML and realization, now as swappable modules
Cloud	Azure, AWS and Google behind one interface, plus a local path
Thin clients	One backend for desktop, web and AR on HoloLens 2

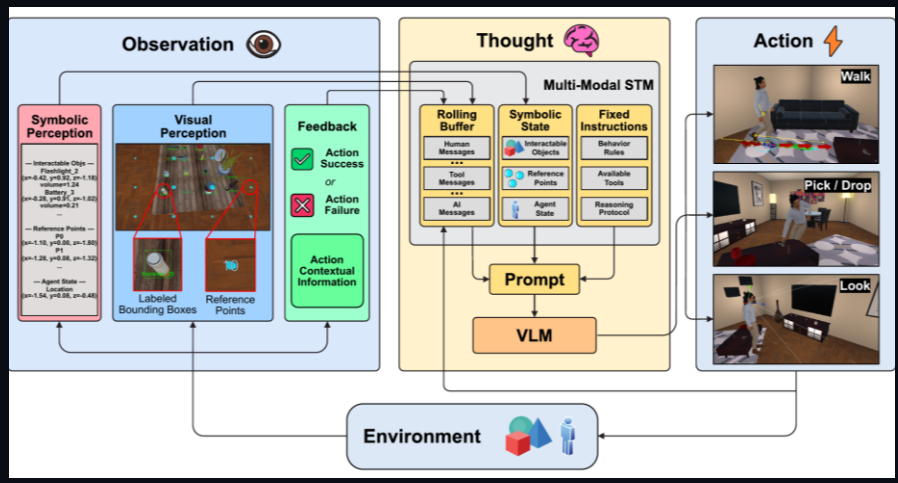
Hartholt et al., Re-architecting the Virtual Human Toolkit, ACM IVA, 2022 (Fig. 2)

04

Virtual Humans in the era of LLMs

What does a language model change, and what does it leave unsolved?





Images: Li, Zhang, Peng & Peiris, IEEE VR 2025 (Fig. 1) · Pecora, Calzolari, Strada & Bottino, arXiv 2026 (Fig. 1)

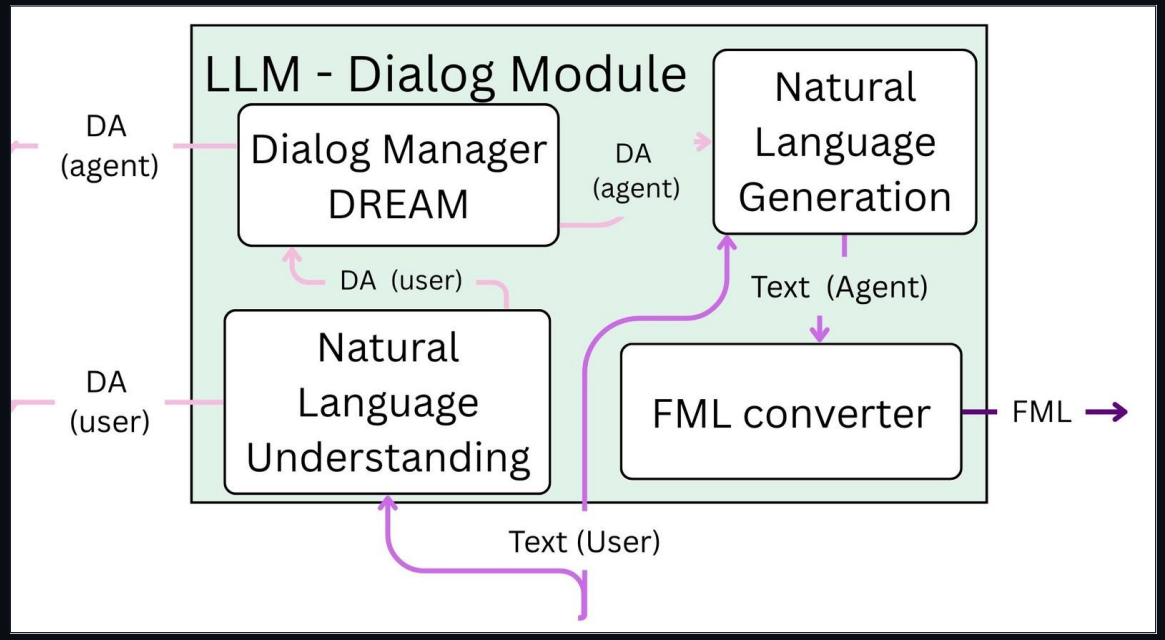
Greta before and after LLMs

	INPUT	INTENT PLANNER	UTTERANCES		BEHAVIOR PLANNER		REALIZER
Greta 2005, SAIBA only	Automated Speech Recognition	Dialogue Manager or script (authored)	Authored by hand	FML	Lexicon picks the signals	BML	Animation, TTS, lip-sync
Greta 2.0 2025	Automated Speech Recognition	NLU and an RL manager pick the dialog act (DA)	The LLM writes them based on DA	FML	Lexicon picks the signals	BML	Animation, TTS, lip-sync

■ changed in Greta 2.0 · ■ the same

Poggi et al., Greta. A believable embodied conversational agent, 2005 · Bevacqua et al., AAMAS 2009 · Saga, Galland, Younsi & Pelachaud, Greta 2.0, ACM IVA 2025

Greta 2.0: dialog acts steer the LLM



Natural Language Understanding	An LLM, prompted with definitions and examples, labels the user's act
Dialogue Manager	A hierarchical RL manager picks the agent's next act
Natural Language Generation	The act, its definition and examples go into the LLM prompt

> User act: **Sharing personal information**
> Agent act: **Asking about current emotions**

Saga, Galland, Younsi & Pelachaud, Greta 2.0: social interactive agent system, optimized for neural network integration, ACM IVA, 2025 (Fig. 4)

Two new problems

Latency

Humans	a few hundred ms between turns
Pipeline	STT, LLM, TTS, animation: ≈ seconds

Research fixes:

Predict the turn change
Inserting "fake" thinking feedbacks
New decision pipelines with Jev-like models

From language to action

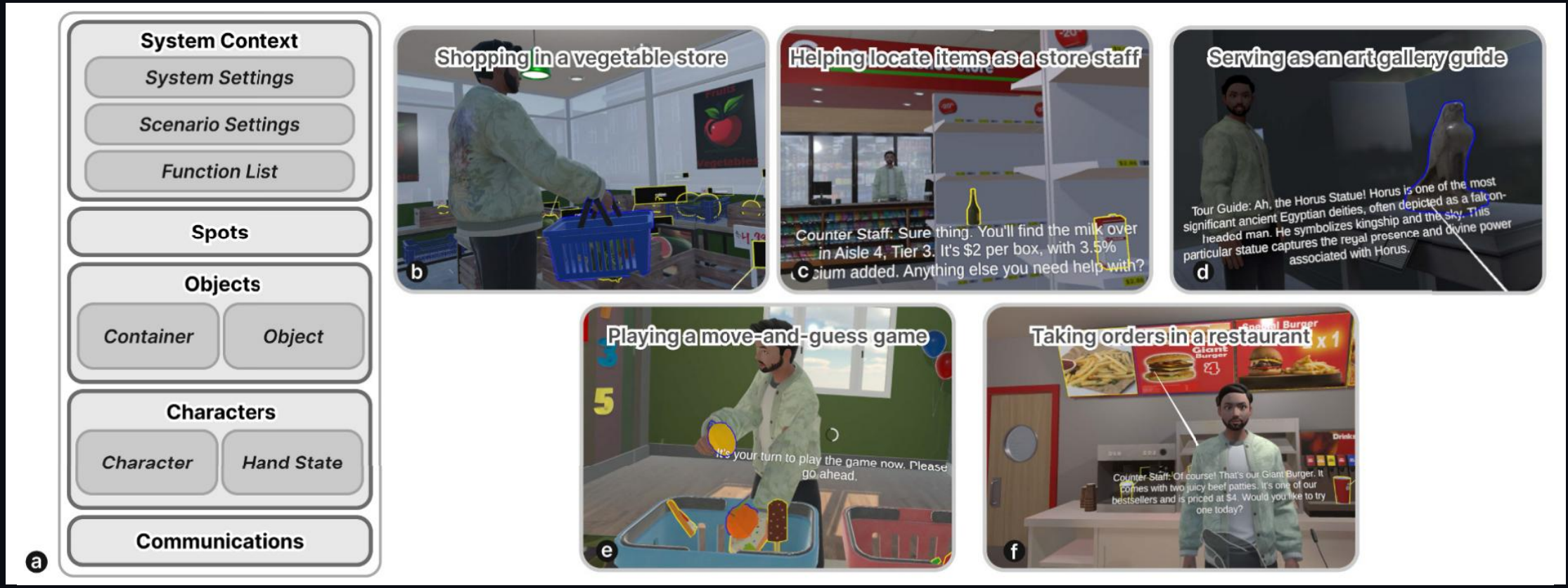
LLM	writes free text: “I wave and smile”
Body	needs commands it can perform, in time with the words

Answer: a fixed list of actions and a structured output

```
{"speech_text": "Hi! …", "action": "WAVE",
 "emotion": "happy"}
```

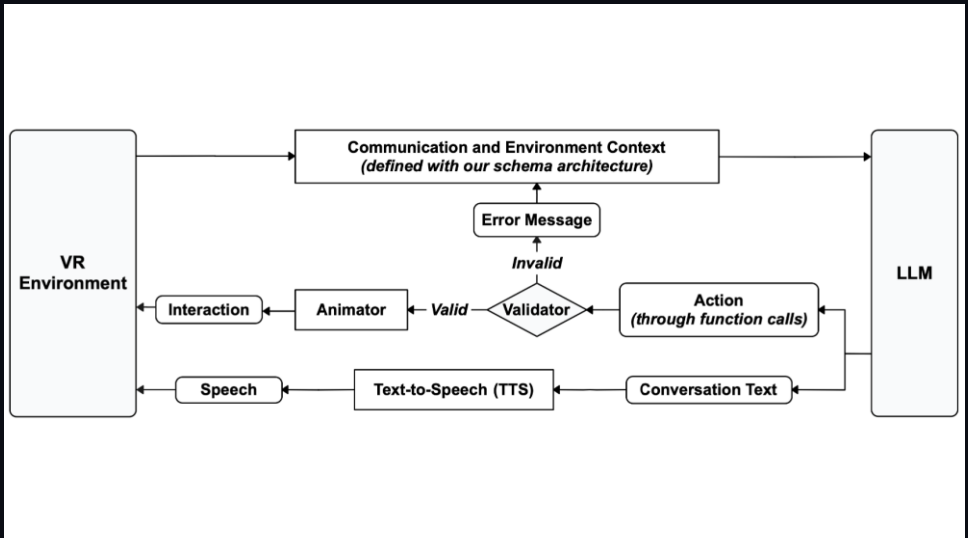
Turn gap: Stivers et al., PNAS 2009 · Fixes: Saga et al., Greta 2.0, ACM IVA 2025 · Action: Li, Mostajeran et al., Frontiers in VR 2026

Li et al.: language-based Situated Interaction



Z. Li, H. Zhang, C. Peng and R. Peiris, "Exploring Large Language Model-Driven Agents for Environment-Aware Spatial Interactions and Conversations in Virtual Reality Role-Play Scenarios," 2025 IEEE Conference on Virtual Reality and 3D User Interfaces (VR) (Fig. 1)

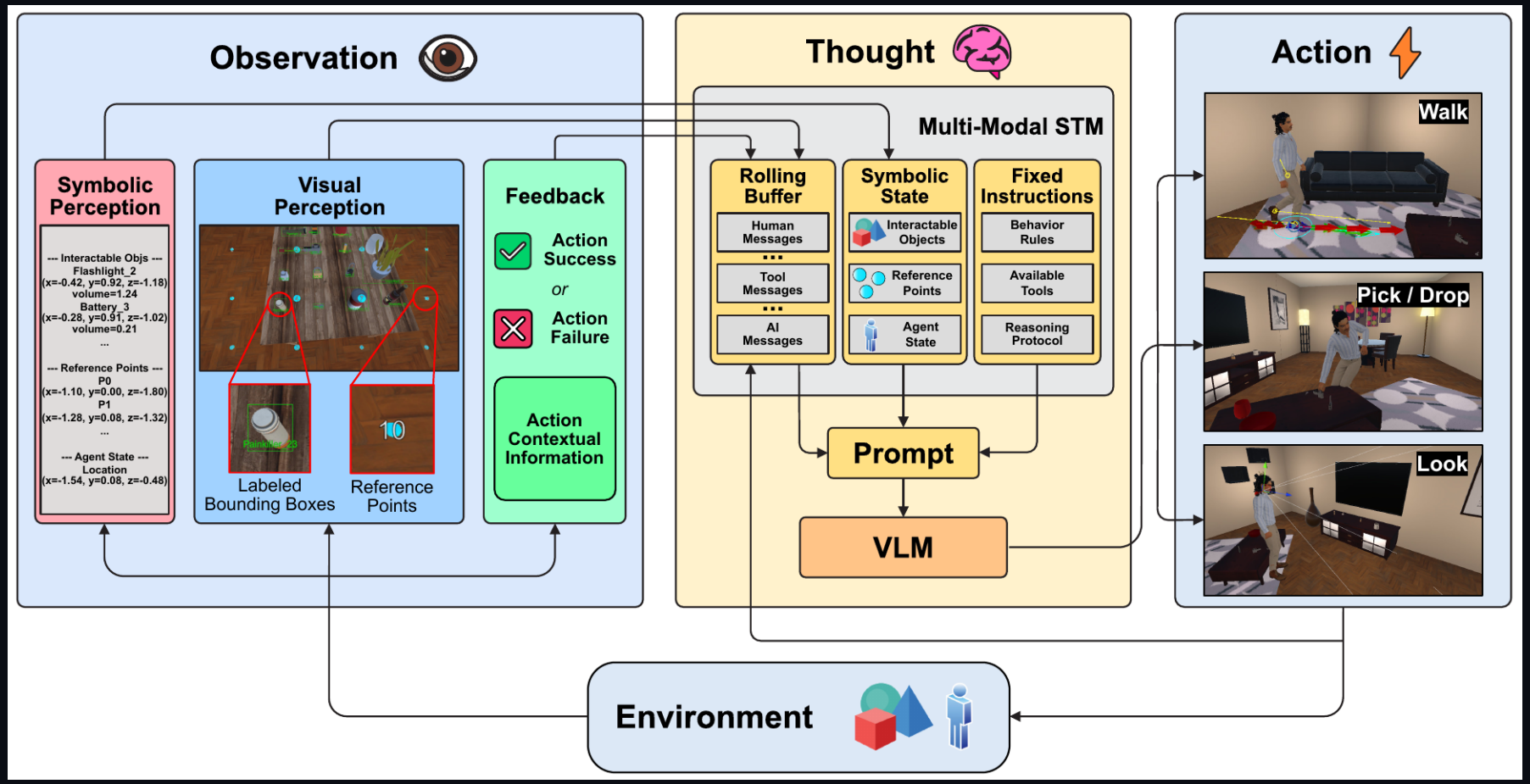
Li et al.: the Architecture



1. **USER** points at the coffee and asks the price; both become text for the LLM
   `<Player's right hand points at Coffee (hot)>`
   `Customer: "Hello! How much is this?"`
2. **LLM** decides to pick the cup up and calls a function
   `<Avatar's left hand grabs Coffee (hot)>`
3. **ENGINE** checks the call and refuses: the cup is out of reach; the error goes back to the LLM
   `<Error: out of your interaction field. Move to a closer spot.>`
4. **LLM** corrects itself: walks to a spot near the cup, then grabs it
   `<Avatar moves to Counter Desk Spot>`
   `<Avatar's left hand grabs Coffee (hot)>`
5. **LLM** answers in words, sent to text-to-speech
   `Server: "Oh! This hot coffee is $5."`

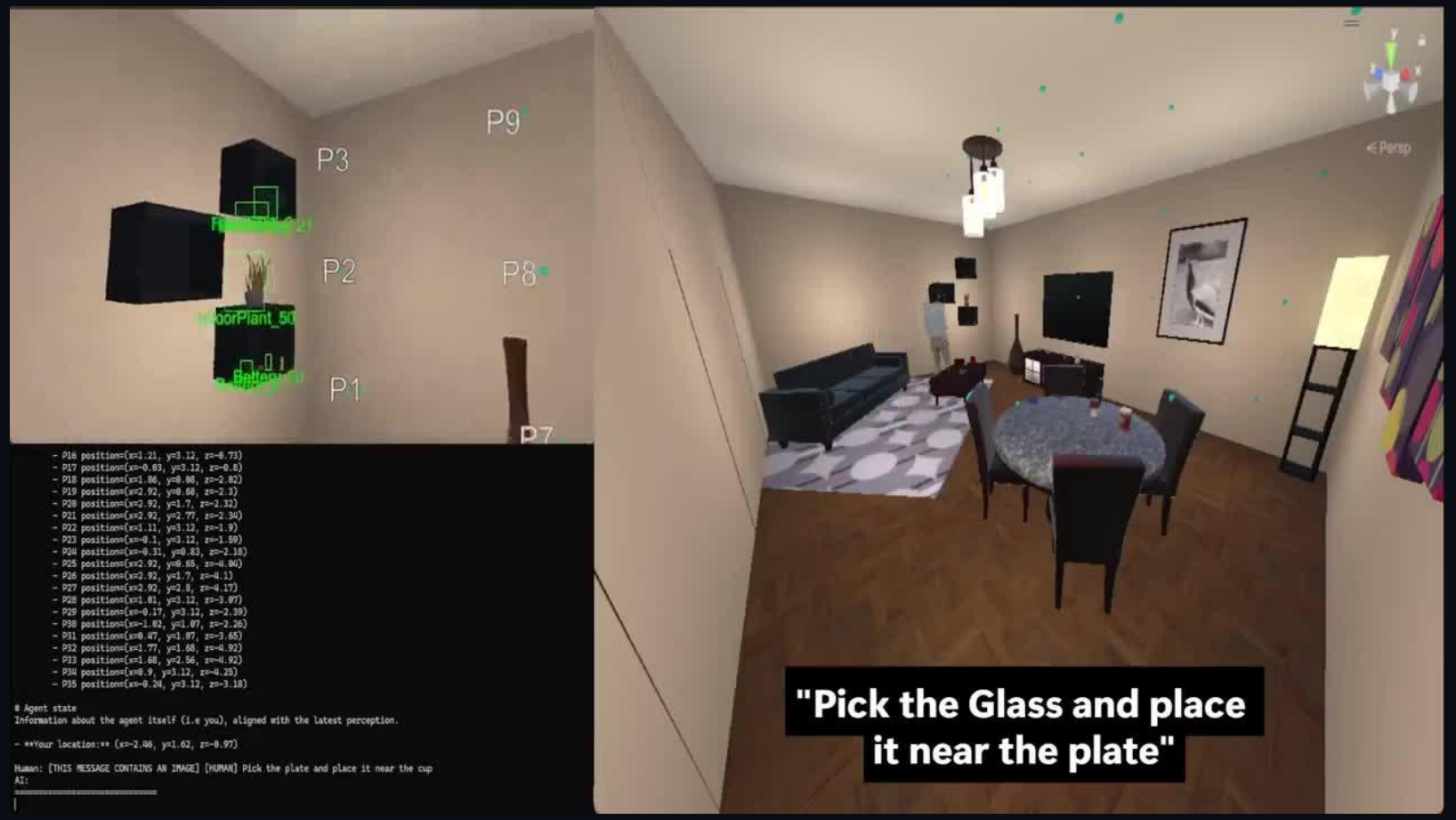
Z. Li, H. Zhang, C. Peng and R. Peiris, "Exploring Large Language Model-Driven Agents for Environment-Aware Spatial Interactions and Conversations in Virtual Reality Role-Play Scenarios," 2025 IEEE Conference on Virtual Reality and 3D User Interfaces (VR) (Fig. 2, Table 1)

Language-based Agency: A.D.A.M.O.

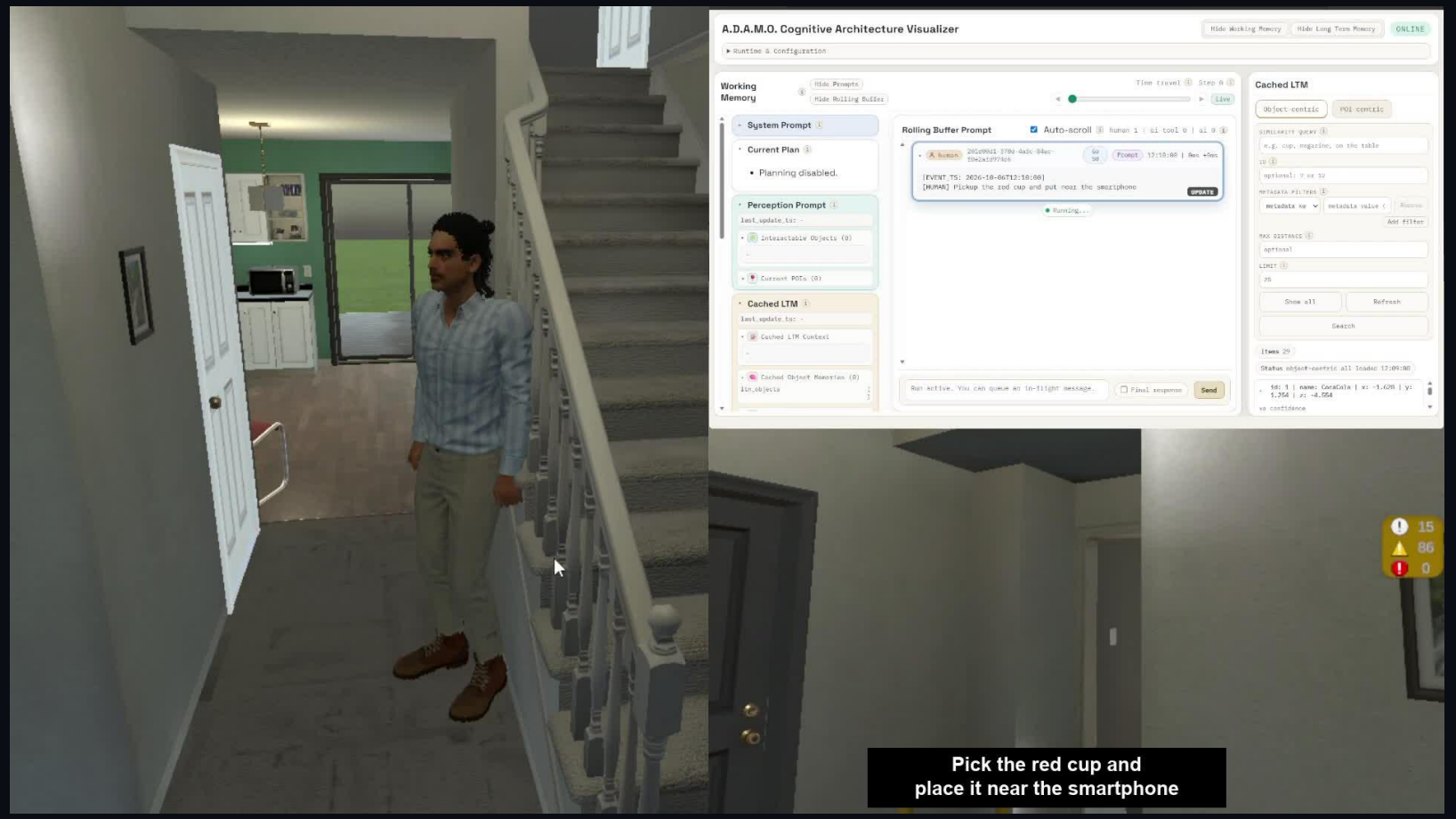


Pecora, Calzolari, Strada & Bottino, A.D.A.M.O.: a visual-symbolic framework for virtual humans, arXiv:2609.35463, 2026 (Fig. 1)

A.D.A.M.O. demo v1



A.D.A.M.O. demo v2



From theory to practice

	Goal	You build	From today
Day 1	The body on stage The speaking body	Character, skeleton, blendshapes; idle breathing and blinking Talk layer, gaze that follows you, lip-sync with visemes	Thalmann: appearance, motion SAIBA realizer; Max's gaze
Day 2	Brain and voice The bridge The voice loop	One LLM call, Whisper for the ears, a TTS voice A FastAPI contract, Unity calls it: the first round trip Push-to-talk, one spoken turn, measure the latency	Greta 2.0: an LLM writes the words VHToolkit: services The turn gap problem
Webinar	The talking character	The LLM chooses a gesture and an emotion with its answer	Li et al.: action inventory

Day 1 in Unity; day 2 starts in Python, then joins the two. The webinar is online.

By the end: a functional ECA

DAY 2
You speak
Push-to-talk in Unity

→

DAY 2
Ears
Whisper turns audio into text

→

DAY 2 · WEBINAR
Brain
An LLM writes the answer, a gesture, an emotion

→

DAY 2
Voice
TTS turns the answer into audio

→

DAY 1 · WEBINAR
Body
Lips, gaze, gesture and face, in sync

BEFORE WE START

INSTALL

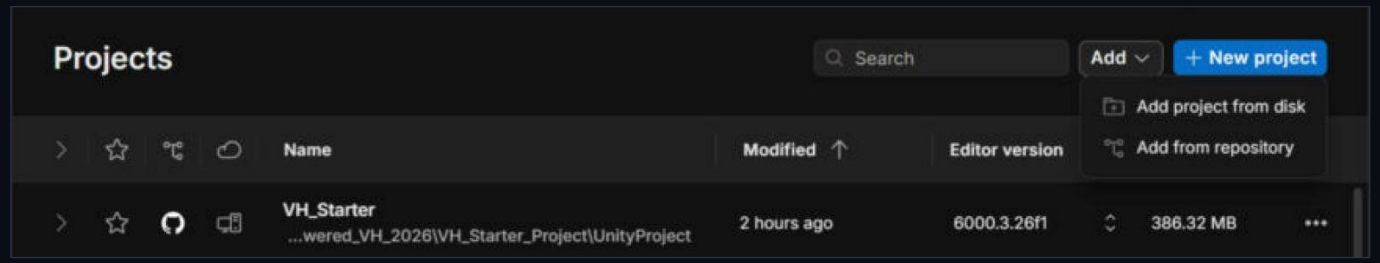
Unity	Hub + Editor 6000.3.26f1 (Unity 6.3)
Python	3.12.x, added to PATH
Git + editors	Git · suggested: Rider (C#), VS Code (Python); any IDE is fine
Ollama	app installed, signed in (free)

DOWNLOAD

Course material	the course template (Unity + Python)
OVRLipSync	29.0.0 from Meta, do not import it yet
Speech models	Whisper small, Piper voice → models/

Get the starter project

1. Clone or download the starter repository
2. Unity Hub → **Add project from disk** → **UnityProject**, editor **6000.3.26f1**
3. Wait for the first import (a few minutes)
4. Console free of **red** errors (yellow avatar warnings are fine)



Unity Hub → Projects → **Add** → **Add project from disk** → pick **UnityProject**. Editor 6000.3.26f1.

What is in the starter project

📦 Complete modules

You never retype them. I explain each one with this map on screen: which part of the system it is, what goes in, what comes out.

▶ LIVE blocks

Marked **// ===== LIVE-n**. We write them together: I type, you type along. A few lines that **connect two parts** of the system.

✎ TURN blocks

Marked **// ===== TURN-n**. You write them, from the hint in the comment. Same kind of connection as the LIVE before it.

Assets/

```
Assets/
 ├── Characters/  Camila_URP.prefab
 ├── Animations/  Mixamo/  VHToolkit/
 ├── Scripts/     every course script
 ├── Settings/    URP
 ├── Audio/       visemes_debug.wav
 │                transitions_debug.wav
 └── Scenes/  Plugins/   (empty)
to_import/      VisemeDriver.cs
```

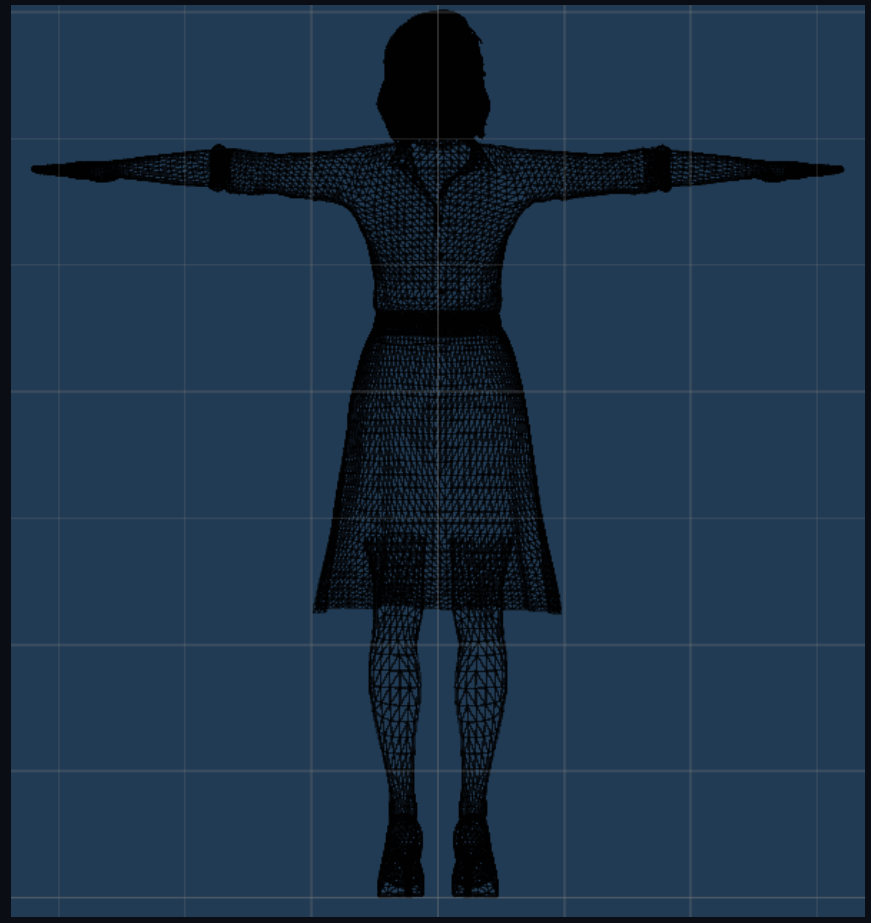
Until a block is written, a **placeholder** keeps the project running: you can always press Play.

06

Anatomy of a humanoid character

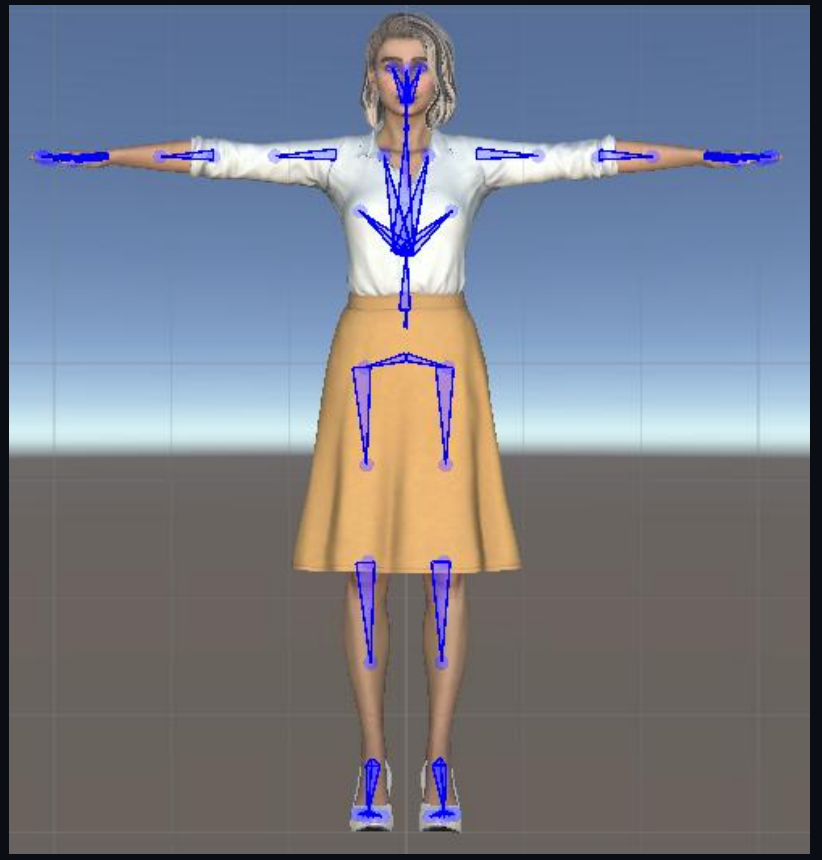
What is a character made of?

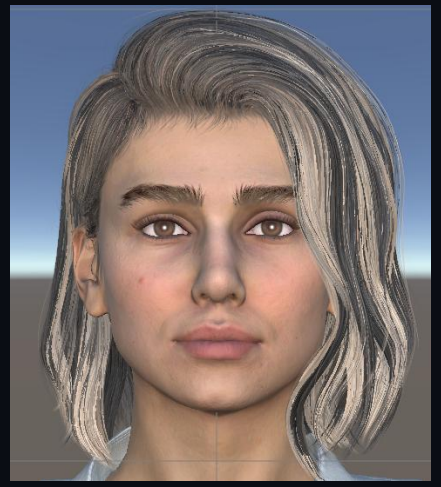
How does a face move without bones?



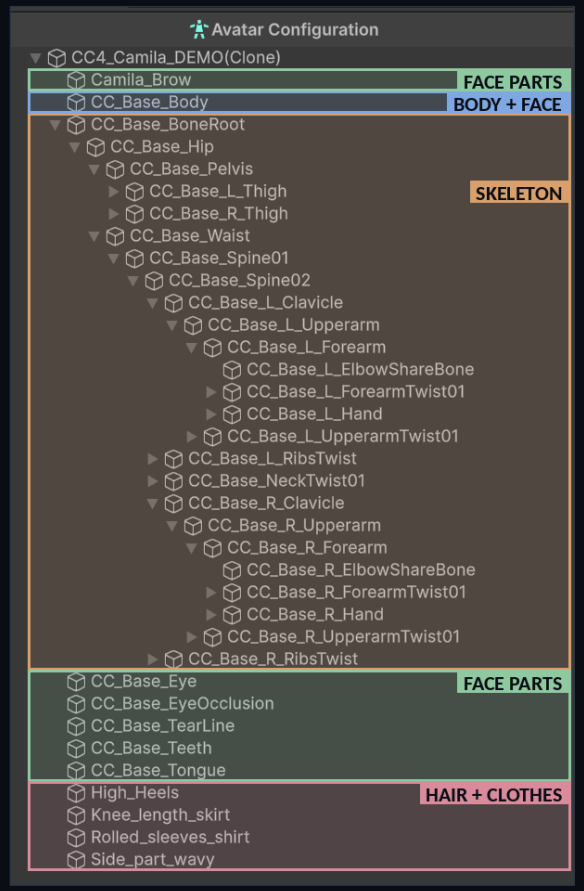
Components of a 3D humanoid model

1. **Mesh**
   The surface: thousands of triangles
2. **Skeleton**
   A hierarchy of bones, hips to head
3. **Skinning**
   Weights bind each vertex to its bones
4. **Materials**
   Shaders and textures: skin, cloth, hair
5. **Blandshapes**
   Blendshapes for muscle/organic movements without bones





Meet the course character

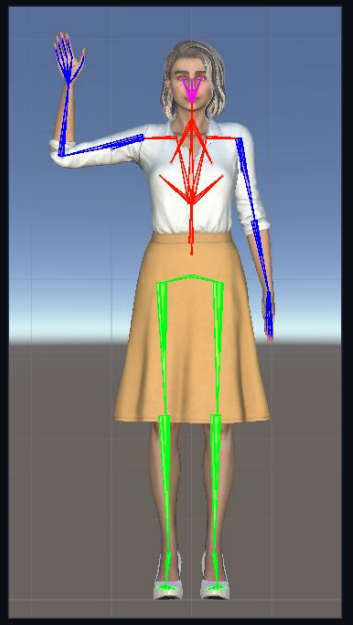


Camila, a Character Creator 4 character · Assets/Characters/Camila_URP.prefab

Reading the tree

`CC_Base_BoneRoot`	The skeleton from **CC_Base_Hip** up; twist bones help the skinning
`CC_Base_Body`	Body **and face** in one mesh: **152 blendshapes**
`Eye · Teeth · Tongue`	Face parts, with EyeOcclusion, TearLine and Camila_Brow: each has its own copy of many shapes
`Hair · clothes`	Side_part_wavy, shirt, skirt, heels; hair and skirt move with **Cloth**

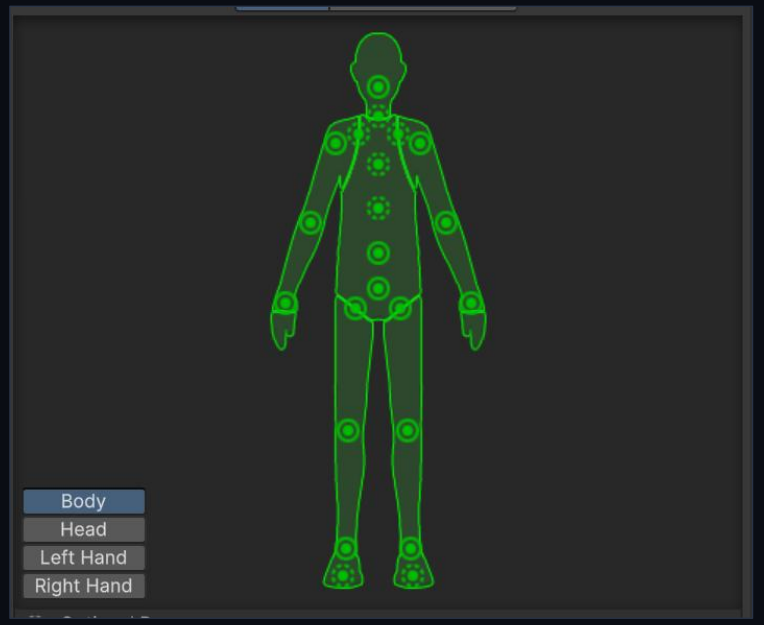
The skeleton: humanoid rig and mapping



1 · The bones in the scene



2 · Avatar: our bones mapped to standard slots



3 · Configure: all green, the rig is valid

Unity Humanoid Avatar

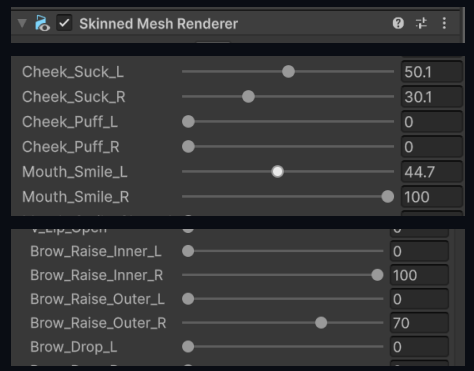
15 required slots, up to about **55**: any humanoid clip can be retargeted

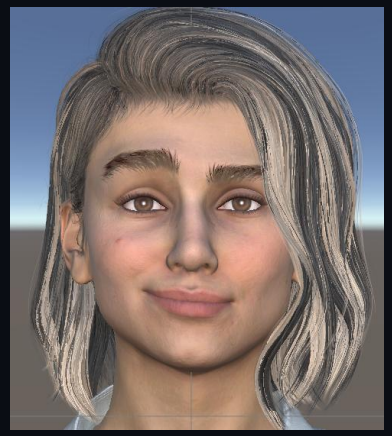


The face: blendshapes



Neutral: all weights 0





Result: the weights mixed

Usual shape groups

Eyes	blink, squint, look
Brows	raise, lower
Mouth	open, close, smile, pucker
Jaw	open, sideways

A stored shape of the same mesh

Each blendshape (morph target) is an alternative version of the face, for example “mouth open as for A”. A weight from **0 to 100** blends neutral and shape; many mix at once.

Inspect the blendshapes

```
// FaceShapes.cs: one helper for every face script
public static List<ShapeRef> Find(faces, "Eye_Blink")
//   -> Eye_Blink_L and Eye_Blink_R on EVERY face mesh
//      (skin, lashes, eye occlusion, tear line, brows)

// BlendShapeLister.cs
[SerializeField] SkinnedMeshRenderer[] faces;
void Reset() => faces = GetComponentsInChildren<...>();
[ContextMenu("Print blendshapes")]  void Print()
```



A shape moved on **one mesh only**: skin and brows no longer match

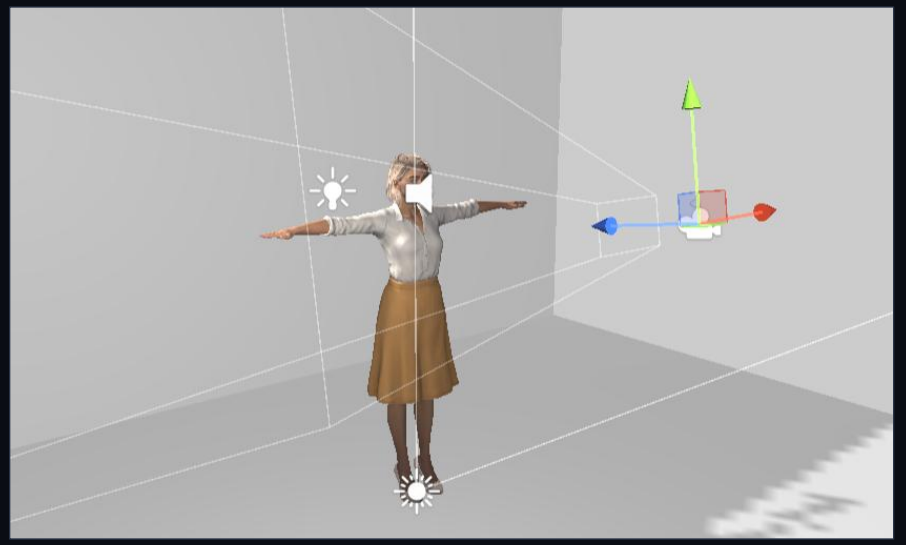
1. Add **BlendShapeLister** to the character root: **Faces** fills itself
2. Component menu → **Print blendshapes**
3. Paste the Console output in **blendshapes.txt**

Mark the **mouth** (**V_***) and **eyelid** (**Eye_Blink**) names: lip-sync and blink will own them.

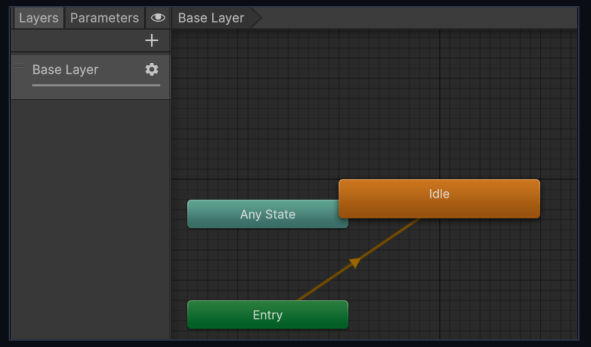
07

Bringing the character to life

How do we make it look alive?





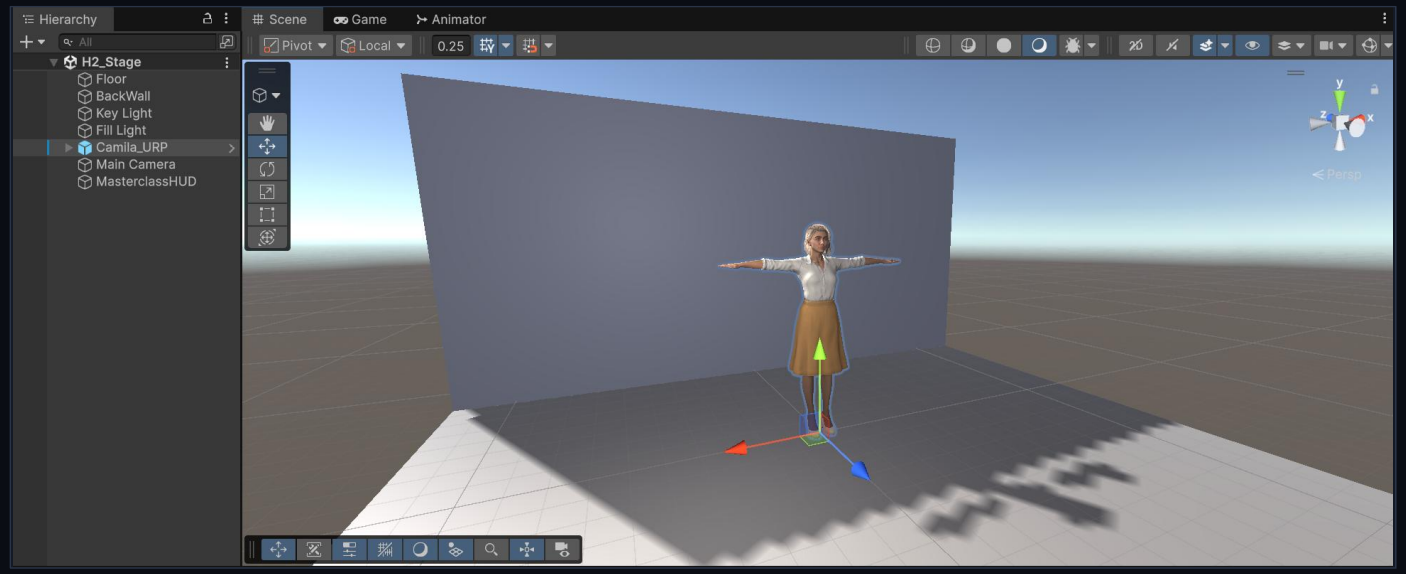


The system we build: today, the body



Put the character on the stage

1. **File → New Scene → Basic (URP)**, save as **Scenes/Stage.unity**
2. Drag **Characters/Camila_URP.prefab** to the origin, rotation **Y = 180**
3. Animator: **Avatar** filled, **Apply Root Motion** off
4. FBX → **Rig** → Configure: every slot green; **Left/Right Eye** and **Jaw** mapped



Frame the conversation

Three rules

Eye level: camera at the character's eyes, about 1.5 m

Chest up: head and chest fill the frame, a small margin above the head

Narrow lens: field of view 30–40°: wide angles distort faces

The stage

Object	Setting
Floor	3D Object → Plane, neutral material
Back wall	Cube, scale (6, 3, 0.2), at z = 2
Light	Directional, soft shadows; optional fill
Main Camera	Position (0, 1.5, −1.6), rotation 0, FOV 35

Adjust until head and chest are framed in the Game view, then save the scene.

Idle life: the antidote to the statue

One blink

Eyes open	Close	Closed	Open	Eyes open
wait 2–6 s, random	~60 ms		~100 ms	next interval

Breathing

An idle clip moves chest and shoulders

Eye saccades

Quick glances instead of a fixed stare

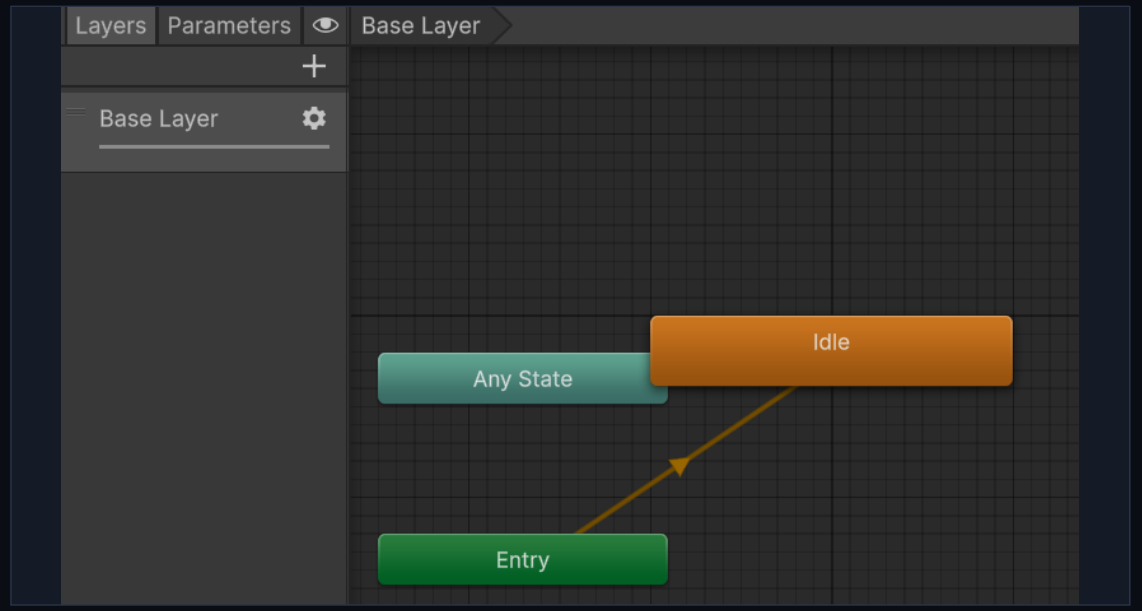
“In nature there exists no motionless character" (Thalmann)

Example: an idle character with blinks and saccades (WebGL)

Blink rate: Bentivoglio et al., Movement Disorders 12(6), 1997 (17/min at rest, 26/min in conversation) · Fast close, slower open: Trutoiu et al., ACM TAP 2011
Quote: Magnenat-Thalmann & Thalmann, Virtual humans: thirty years of research, what next?, The Visual Computer 21(12), 2005 (§4.5)

The idle animation: an Animator Controller

1. Animations → Create → **Animator Controller** → **VH_Controller**
2. Drag **Mixamo/Idle** in: the orange **default state**
3. Idle.fbx → Rig **Humanoid**; Animation **Loop Time** ✓ → Apply
4. Character root → Animator → **Controller** = VH_Controller
5. **Play**: Camila breathes and shifts slightly



Humanoid

The Mixamo clip retargets to Camila's bones through the Avatar

Loop Time

The idle never ends: no frozen pose at the last frame

BlinkController 1/4: what it is given

```
public class BlinkController : MonoBehaviour
{
    [SerializeField] SkinnedMeshRenderer[] faces;
    // fragments of the blink names, from your list
    [SerializeField] string[] blinkShapesContain = { "Blink" };
    [SerializeField] Vector2 interval = new Vector2(2f, 6f);
    [SerializeField] float closeTime = 0.06f;
    [SerializeField] float openTime = 0.10f;

    List<ShapeRef> shapes = new List<ShapeRef>();
```

All the **face meshes**, not one

Name fragments, not indices: indices differ between characters

Human numbers: every **2–6 s**, **60 ms** close, **100 ms** open

BlinkController 2/4: find the shapes, move them

```
void Start()
{
    foreach (var part in blinkShapesContain)
        shapes.AddRange(FaceShapes.Find(faces, part)); // + warning
    if (shapes.Count == 0) { enabled = false; return; }
    StartCoroutine(BlinkLoop());                    // -> LIVE-1
}

IEnumerator Animate(float from, float to, float duration)
{
    for (float t = 0f; t < duration; t += Time.deltaTime)
    {   SetAll(Mathf.Lerp(from, to, t / duration)); yield return null; }
    SetAll(to);
}
```

A missing name prints a **warning**, never a crash

Animate: one step per frame, ends exactly on the target

Start calls **BlinkLoop**: the part we write

LIVE-1: the loop that drives the face

```
IEnumerator BlinkLoop()
{
      while (true)
      {
          float wait = Random.Range(interval.x, interval.y);
          yield return new WaitForSeconds(wait);
          yield return Animate(0f, 100f, closeTime);    // close
          yield return Animate(100f, 0f, openTime);     // open
      }
}
```

Connects a **timer** to the **face**

A **coroutine**: it waits without blocking the frame

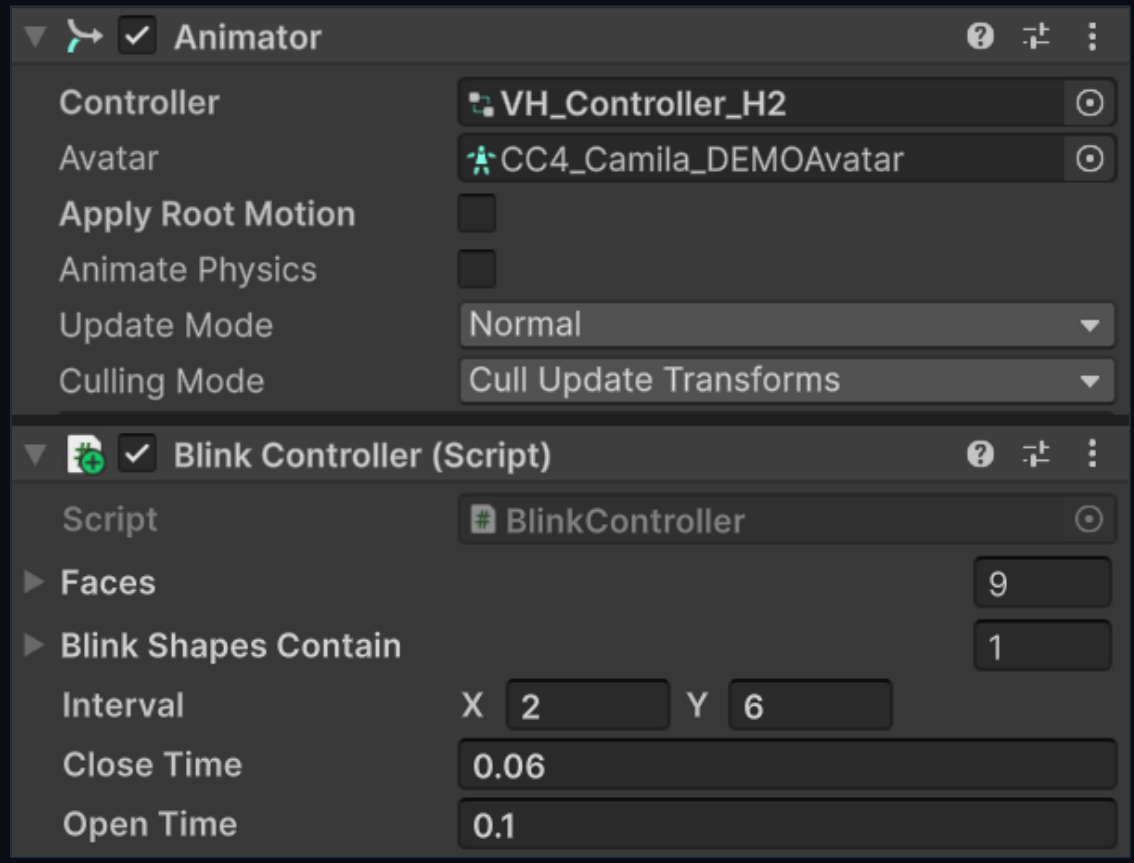
A random wait, **never a rhythm**

BlinkController 4/4: wire the blink

1. Add **BlinkController** to the character root
2. **Faces**: Copy Property from BlendShapeLister, Paste
3. **Blink Shapes Contain** → Element 0 = **Eye_Blink**
4. **Play**: breathing, and a blink every 2–6 s

Breathes but does not blink?

Is LIVE-1 written? Read the Console: a missing name prints a warning.



Finish idle and blink

Fix, then experiment with the interval and the speed. What looks natural? What looks creepy?

Problem	Fix
No blink, no warning	LIVE-1 not written, script on the wrong object, or Faces empty
Warning “not found”	The shape has a different name: use a unique fragment from your list
The script runs, nothing moves	Something else writes the same shapes every frame (an animation clip with blendshape curves): remove the curves or exclude the face
Character slides during idle	Turn off Apply Root Motion on the Animator

08

Looking at you: gaze

Where should the character look?

Why does a fixed stare feel wrong?

Look at the user

head and eyes toward the camera

Small glances

saccades: quick, random, tiny

Never all the way

the head turns a little, the eyes do the rest

Gaze: three jobs

Attention

Looking at the user says “I am talking to you”

Life

Small, quick, random glances: a fixed stare looks dead

Comfort

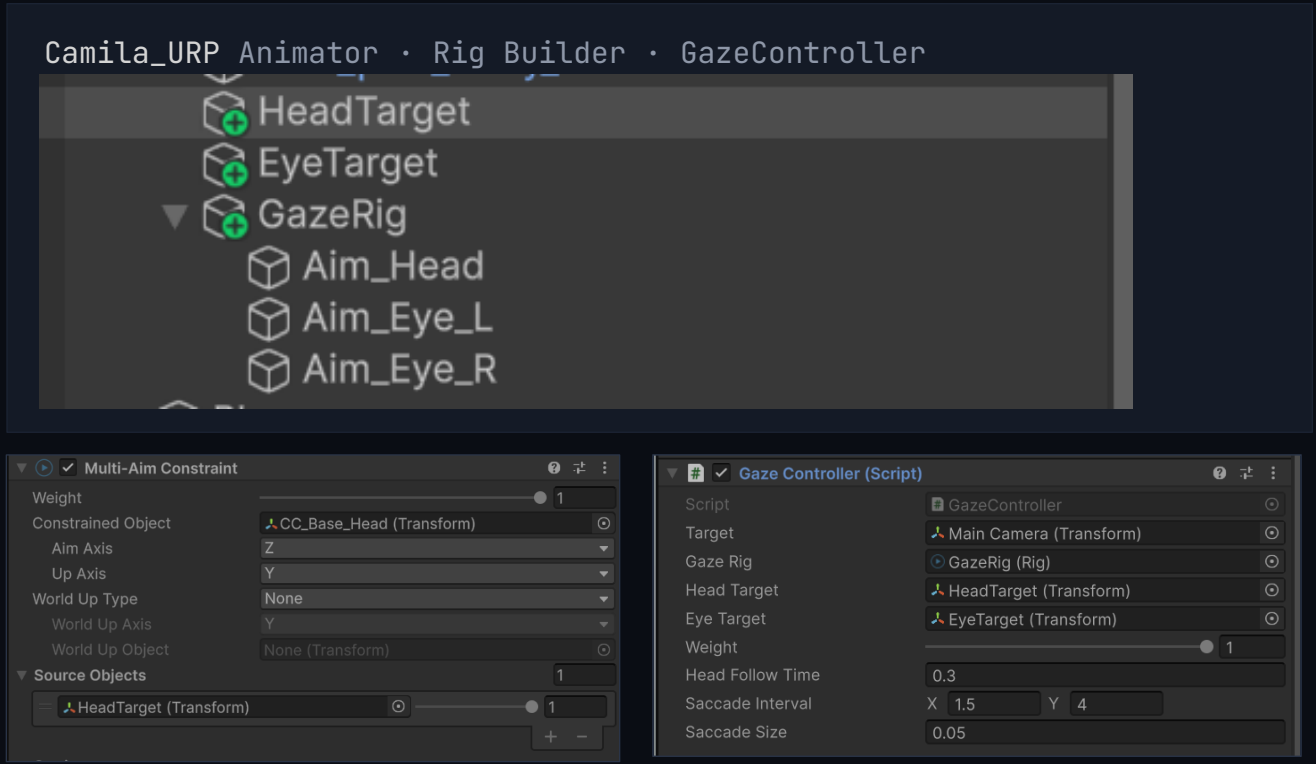
Head turns partly, eyes finish the job: no owl necks

Animation Rigging A rig of **Multi-Aim** constraints: chest, neck and head each turn part of the way, the eyes finish

Same idea as the blink: a constant is robotic, a little randomness is alive

Set up the gaze rig

1. Root → **Animation Rigging → Rig Setup**: Rig Builder + a Rig → rename **GazeRig**
2. Next to GazeRig: empty **HeadTarget** and **EyeTarget**
3. One **Multi-Aim Constraint** per bone: object, source, **Weight**
4. **Aim Axis**: the bone axis out of the face (Move tool, **Local**)
5. **GazeController** on the root: Main Camera, GazeRig, the two targets



GazeController 1/2: the settings

```
public class GazeController : MonoBehaviour
{
    [SerializeField] Transform target;              // the Main Camera
    [SerializeField] Rig gazeRig;                   // the Animation Rigging rig
    [SerializeField] Transform headTarget, eyeTarget;
    [SerializeField, Range(0, 1)] float weight = 1f;
    [SerializeField] float headFollowTime = 0.3f;    // seconds
    [SerializeField] Vector2 saccadeInterval = new Vector2(1.5f, 4f);
    [SerializeField] float saccadeSize = 0.05f;      // metres around the target
```

No IK in the script: the **rig** does it

It only moves **two targets**

Saccades: how often, how far

GazeController 2/2: two targets

```
void Update()
{
    if (Time.time >= nextSaccade)            // every 1.5–4 s: a new offset
    {
        offset = Random.insideUnitSphere * saccadeSize;
        nextSaccade = Time.time
            + Random.Range(saccadeInterval.x, saccadeInterval.y);
    }
    headTarget.position = Vector3.SmoothDamp(headTarget.position,
        target.position, ref headVelocity, headFollowTime);   // head: smooth
    eyeTarget.position = target.position + offset;           // eyes: jumps
    gazeRig.weight = Mathf.MoveTowards(gazeRig.weight, weight,
        2f * Time.deltaTime);                                 // fade in/out
}
```

HeadTarget glides to the camera: the head stays steady

EyeTarget jumps: the saccade moves the eyes only

SetWeight(0): the clips take the head back

Try it: does it look at you?

1. Play mode: the character looks at the camera
2. Move the camera: chest, head and eyes follow
3. Watch the eyes: small jumps every few seconds

Troubleshooting

Symptom	Fix
Nothing happens	Rig Builder missing on the root, or GazeRig not in its Rig Layers
Looks at the ceiling or sideways	Wrong Aim Axis on that constraint
Head turns, eyes do not	Eye constraints not on EyeTarget, or wrong eye axis
Neck twists too far	Lower the head weight
Nervous eyes	Longer saccade interval, smaller size

What works now: idle, blinking and gaze



✓ **Framed**
Head and chest in the Game view, eye-level camera

✓ **Breathing**
The idle loops, no sliding

✓ **Blinking**
Every 2–6 s, at a natural speed

✓ **Looking**
Head and eyes follow the camera, small glances

Next: the Animator, and the character's first API.

09

Talking: a layer for the upper body

How does the body know it is speaking?

Who is allowed to tell it?



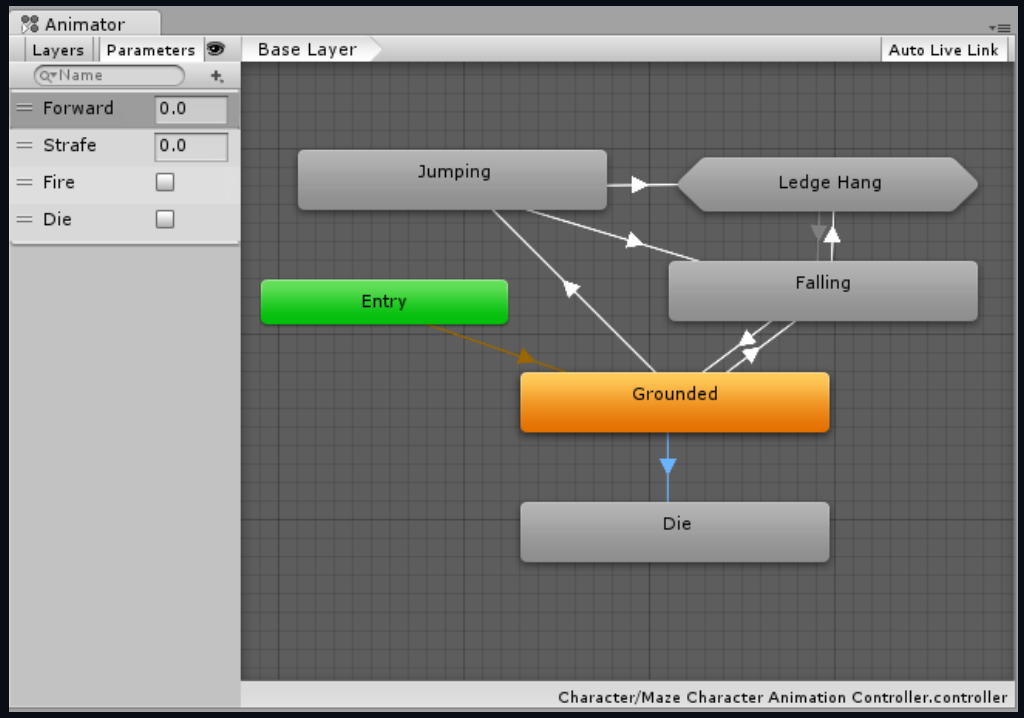
The Animator in one picture

States: what the character is doing now · Idle, Speaking

Parameters: variables set from code · bool Speaking

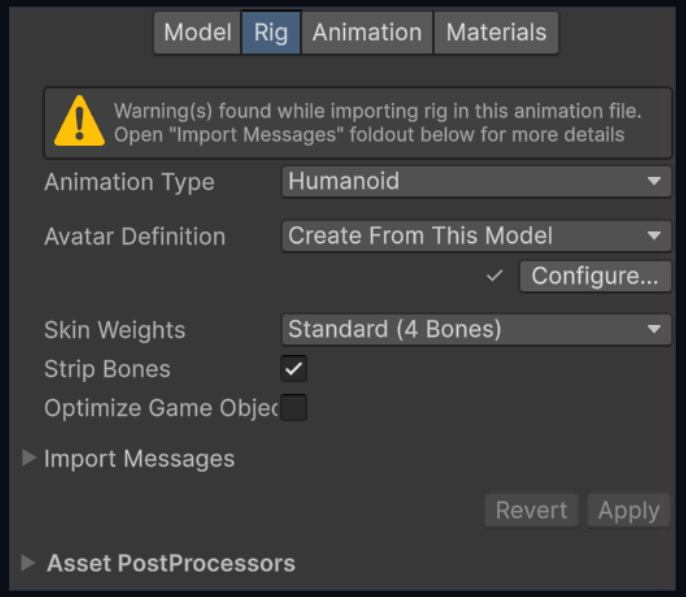
Transitions: arrows with conditions · Speaking == true

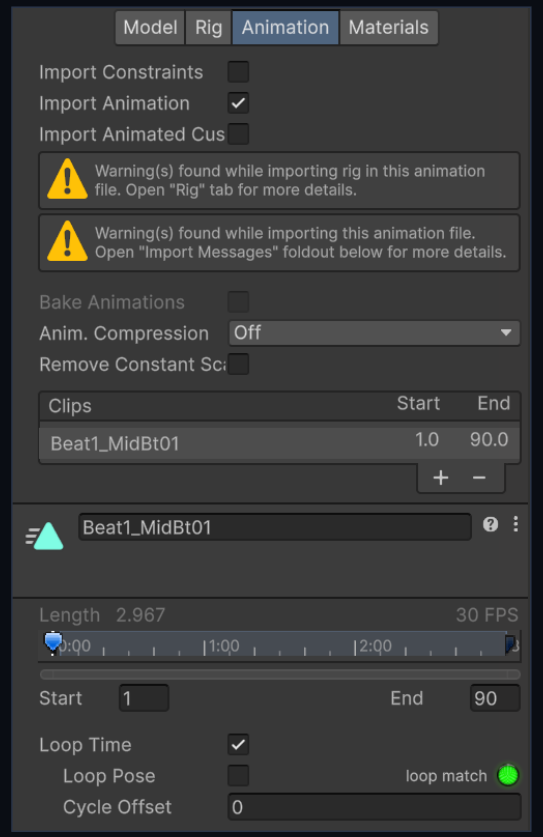
Layers: several animations playing at once, each one can be **masked**



Import the talk clips: one talking loop

1. In **Animations/VHToolkit/** select **all the …_Beat** clips together (Ctrl+click): the settings apply to all
2. Rig: **Humanoid**, Avatar Definition = **Create From This Model**
3. **Bake Into Pose**: root rotation, position Y and XZ
4. **Apply**

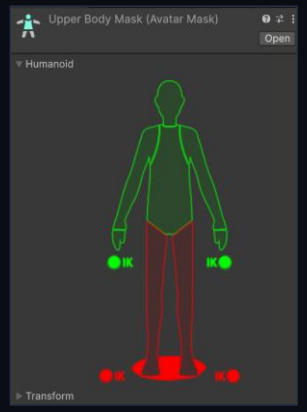




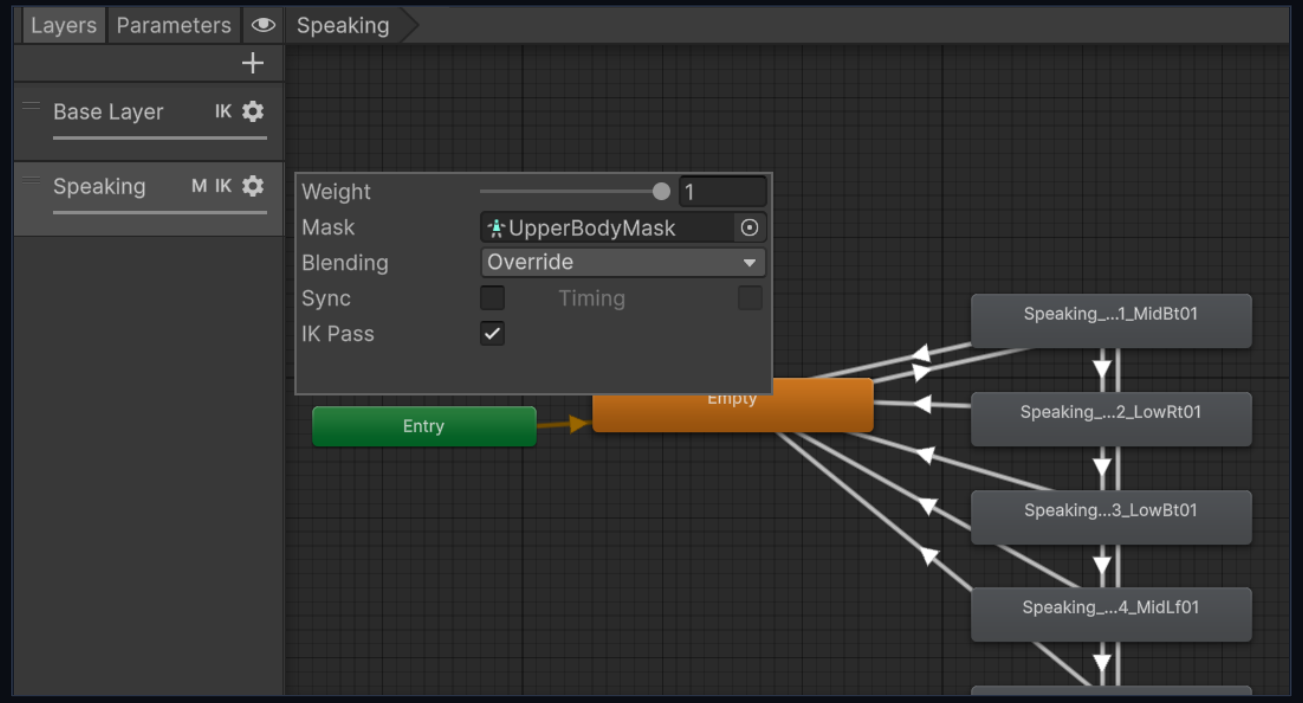
Trick: several beat clips combined together make a single talking loop not repeating the same gesture

Idle below, Speaking on top: a masked layer

1. Create → **Avatar Mask UpperBodyMask**: Humanoid, untick **Root**, **legs**, **foot IK**
2. VH_Controller → Parameters → Bool **Speaking**
3. Layers → **+ Speaking**: Weight 1, Override, Mask = UpperBodyMask
4. In it: **Empty** (default) ⇄ **Speaking** (the beat clip)
5. Transitions on the bool, **Has Exit Time** off, **0.2 s**



UpperBodyMask: green = the layer may move it.



Speaking layer settings (IK Pass is no longer needed). The reference has five beat clips; you build one.

CharacterBehaviorManager: the only door

```
public partial class CharacterBehaviorManager : MonoBehaviour
{
    [SerializeField] Animator animator;
    static readonly int SpeakingHash =
        Animator.StringToHash("Speaking");
    void Reset() => animator = GetComponent<Animator>();

    public void SetSpeaking(bool speaking)
    {
        // ===== LIVE-2 · Body ▸ Animator · Speaking on/off =====
          animator.SetBool(SpeakingHash, speaking);
    }

    [ContextMenu("Test/Speaking On")] void SpeakOn() => SetSpeaking(true);
    [ContextMenu("Test/Speaking Off")] void SpeakOff() => SetSpeaking(false);
}
```

Nothing else touches the **Animator**

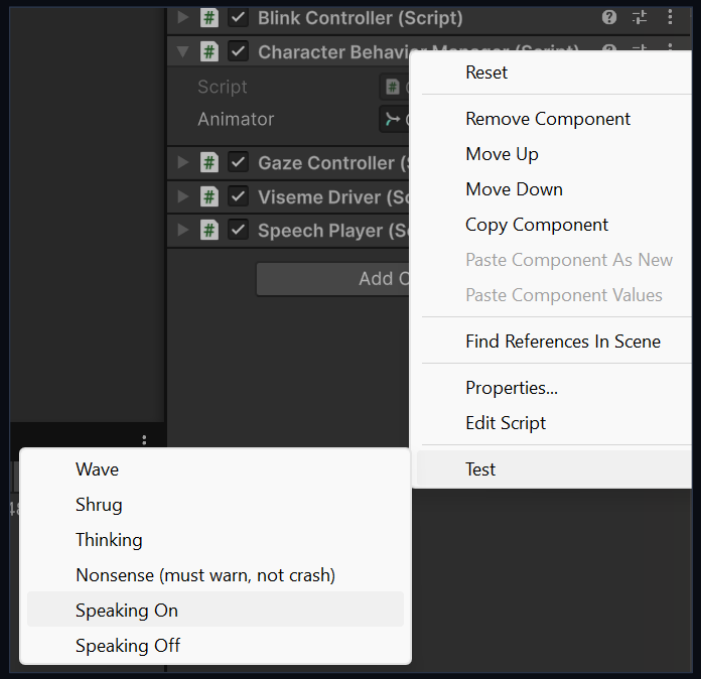
StringToHash once, cached

Speech, network, brain: all ask **this** class

Try it: talk on, talk off

Add **CharacterBehaviorManager** to the root, Play mode, right-click the component → **Test/Speaking On** and **Off**.

Problem	Fix
Nothing happens	LIVE-2 not written, or the controller not assigned to the Animator
The legs move with the talking	The Speaking layer has no UpperBodyMask
Snaps or pops	Has Exit Time still on, or duration 0
The character slides	Bake Into Pose, or Apply Root Motion off





Right-click the component → **Test** → **Speaking On**: arms and torso talk, legs stay in idle.

10

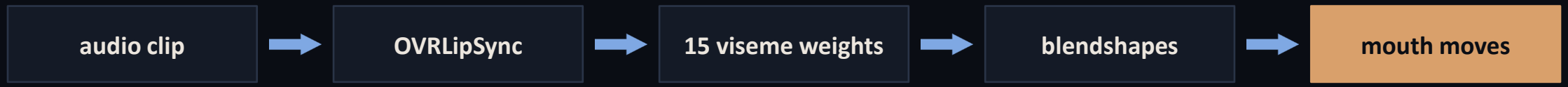
Voice and lips

How does a sound become a mouth shape?

Which of your blendshapes does each one drive?



From audio to a moving mouth



Phonemes → visemes: many to one

Phonemes are the sounds of a language, dozens of them. **Visemes** are the mouth shapes you can see.

B · P · M → one shape: **closed lips**

Lips from the **audio**, not from the text: any voice, any language, even a recording of you.

The 15 Oculus visemes

sil silence	**PP** **p**ay, **b**ay	**FF** **f**ish, **v**an	**TH** **th**ink	**DD** **d**og, **t**ime
kk **k**ey, **g**o	**CH** **ch**in, **sh**ip	**SS** **s**un, **z**oo	**nn** **n**ow, **l**ip	**RR** **r**ed
aa f**a**ther	**E** b**e**d	**ih** s**i**t	**oh** g**o**	**ou** y**ou**

rest · ten consonant groups · five vowels

Meta, Oculus Lipsync for Unity (course material: Oculus_LipSync.url)

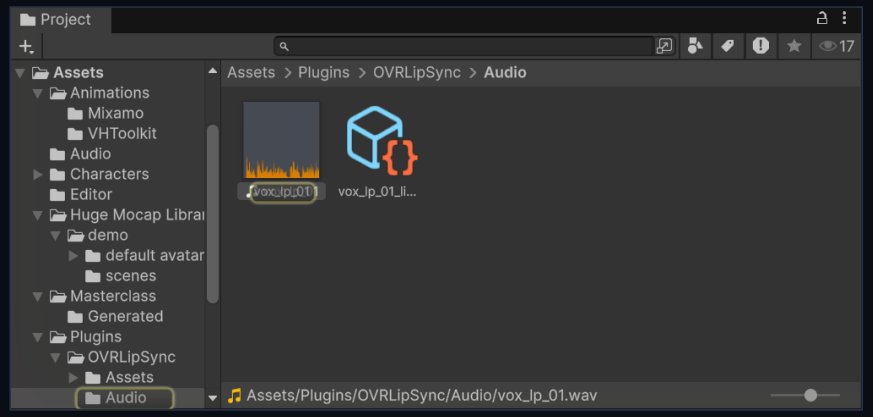
Import OVRLipSync and setup test clip

1. Unzip Meta's download → **OculusLipSync.unitypackage**
2. Assets → Import Package → Custom Package → **Import** all
3. Plugin folder → **Assets/Plugins/OVRLipSync/**
4. Copy **to_import/VisemeDriver.cs** into **Assets/Scripts/**
5. Plugin **Audio/vox_lp_01.wav** → Ctrl+D → **sample_speech** → **Assets/Audio/**

Red error about a UI type? **Oculus.LipSync.asmdef** → references → **+ UnityEngine.UI**

Each student downloads the plugin from Meta: its licence does not allow sharing it

Apple Silicon: the plugin is x86_64 only, the driver falls back to loudness



6. **Delete LipSyncDemo_SetCurrentTarget.cs** (versioning error)

```
Assets\Oculus\LipSync\Scenes\Scripts\LipSyncDemo_SetCurrentTarget.cs(108,43): error CS0103:
```

Lip-sync wiring

AudioSource
plays the clip

→

OVRLipSyncContext
15 viseme weights

→

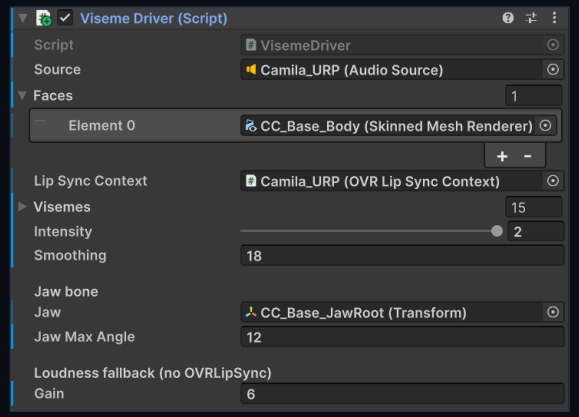
VisemeDriver
mix onto CC4 shapes + jaw

→

Face meshes
V_* shapes, jaw bone

1. Select **Camila_URP** (root) → Add Component → **Audio Source**: Play On Awake off, Spatial Blend 0
2. **Camila_URP** → Add Component → **OVR Lip Sync Context**: Audio Source = it, **Audio Loopback** ✓
3. **Camila_URP** → Add Component → **VisemeDriver**: Source, Faces, Lip Sync Context (drag the **component**), Jaw = **CC_Base_JawRoot**
4. **to_import/VisemePreset.txt** → Ctrl+A, Ctrl+C → **VisemeDriver → Visemes → Paste**

Field	Effect
Gain (context)	Meter peaks at ~0.9, just before it clips
Intensity (driver)	How far the mouth opens
Smoothing (driver)	Higher: snappier
Jaw Max Angle	12°: lower if it gapes



Map the visemes to your blendshapes

Viseme	CC4 mix	Jaw
sil	(rest: no shapes)	0
PP	V_Explosive 100	0
FF	V_Dental_Lip 100	0.05
TH	V_Tongue_Out 60, V_Open 20	0.15
DD	V_Open 30, V_Tongue_Raise 60	0.2
kk	V_Open 40, V_Tongue_Lower 30	0.3
CH	V_Affricate 100	0.1
SS	V_Wide 60	0.05

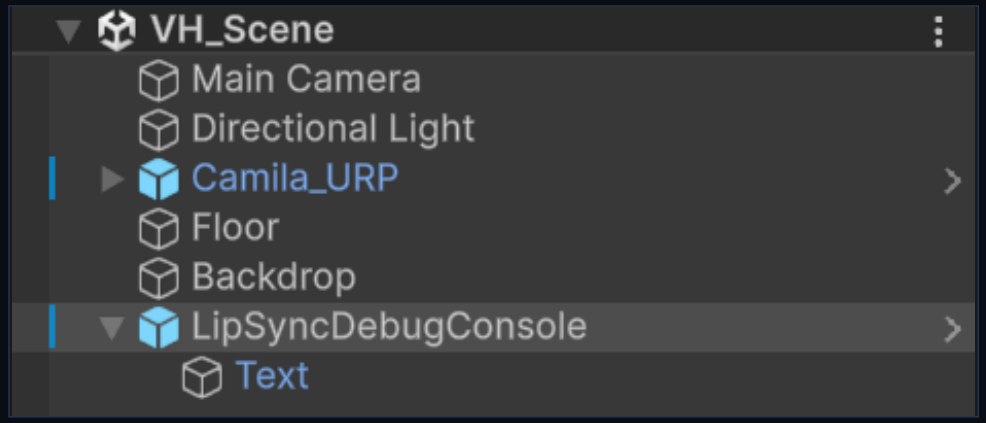
Viseme	CC4 mix	Jaw
nn	V_Lip_Open 50, V_Tongue_Raise 50	0.15
RR	V_Tight_O 50, V_Tongue_Curl_U 40	0.15
aa	V_Open 100	1
E	V_Wide 70, V_Open 40	0.45
ih	V_Wide 60, V_Open 20	0.25
oh	V_Tight_O 80, V_Open 30	0.5
ou	V_Tight 90	0.15

The viseme debugger: see what the plugin hears

1. Project search: **LipSyncDebugConsole** (the plugin's prefab)
2. Drag it into **VH_Scene**, at the root, next to the character
3. **OVR Lip Sync Context** → **Show Visemes** ✓
4. **Play** the sample clip: one row per viseme, more * = stronger
5. Try out the others clips:
   Audio/visemes_debug.wav
   Audio/transitions_debug.wav

Vowels last longest and carry the stress: they open the jaw

Consonants last tens of ms, but a missed PP or FF is what viewers notice





Try it: a sample clip, then tune

AudioSource clip = **sample_speech**, tick **Play On Awake** for this test, press **Play**. Untick it afterwards.

Symptom	Cause and fix
No sound	Audio Loopback off on the context
Sound, mouth does not move	Lip Sync Context empty or wrong type; Faces empty; names not matched (Test/Print mapping)
Mouth barely moves	Context Gain up, or driver Intensity up
Mouth opens too wide	Intensity down, or Jaw Max Angle down
Mouth stuck open at the end	The sil entry must have no shapes



Mid-word: the mouth follows the voice.



Gaze and blink keep running while she speaks.

TURN-1: voice and body together

```
public class SpeechPlayer : MonoBehaviour
{
    [SerializeField] AudioSource source;   // lip-sync listens
    [SerializeField] CharacterBehaviorManager behavior;  // door
    [SerializeField] AudioClip testClip;

    public IEnumerator Download(...)   // Hour 5: from the server

    public IEnumerator Play(AudioClip clip)
    {
        if (clip == null) yield break;
        // ===== TURN-1 · Voice ▸ Body =====
        // play the clip on source; Speaking on for exactly
        // as long as it plays (clip.length + 0.15 s)
    }
}
```

You need: **source.clip**, **source.Play()**, **behavior.SetSpeaking**, **WaitForSeconds**

Not **isPlaying**: it breaks when the editor loses focus

Select **Camila_URP** (root) → Add Component → **SpeechPlayer**:
Source = its **Audio Source**, Behavior = its **CharacterBehaviorManager**, Test Clip = **sample_speech** → ⋮ → **Test/Play test clip**

Module 1 done: the body

The body can speak

- ✓ Idle and blinking
- ✓ A Speaking layer
- ✓ Gaze with saccades
- ✓ Lips in sync with a clip
- ✓ One door: SetSpeaking

Next: the other track

No Unity: Python only

One call to a language model

Speech to text, text to speech

Each stage tested alone

Unity: the body → **FastAPI: the bridge** → **The brain**

What works now: the body "endpoint"



✓ **Talking**
Speaking layer on and off, legs in idle

✓ **Looking**
Gaze follows the camera, small glances

✓ **Lips**
The test clip plays in sync, idle at the end

Thank you

Questions?

Next · **Day 2**: brain and voice in Python, the bridge, the voice loop



Stefano Calzolari · Wed 21 – Fri 23 October 2026